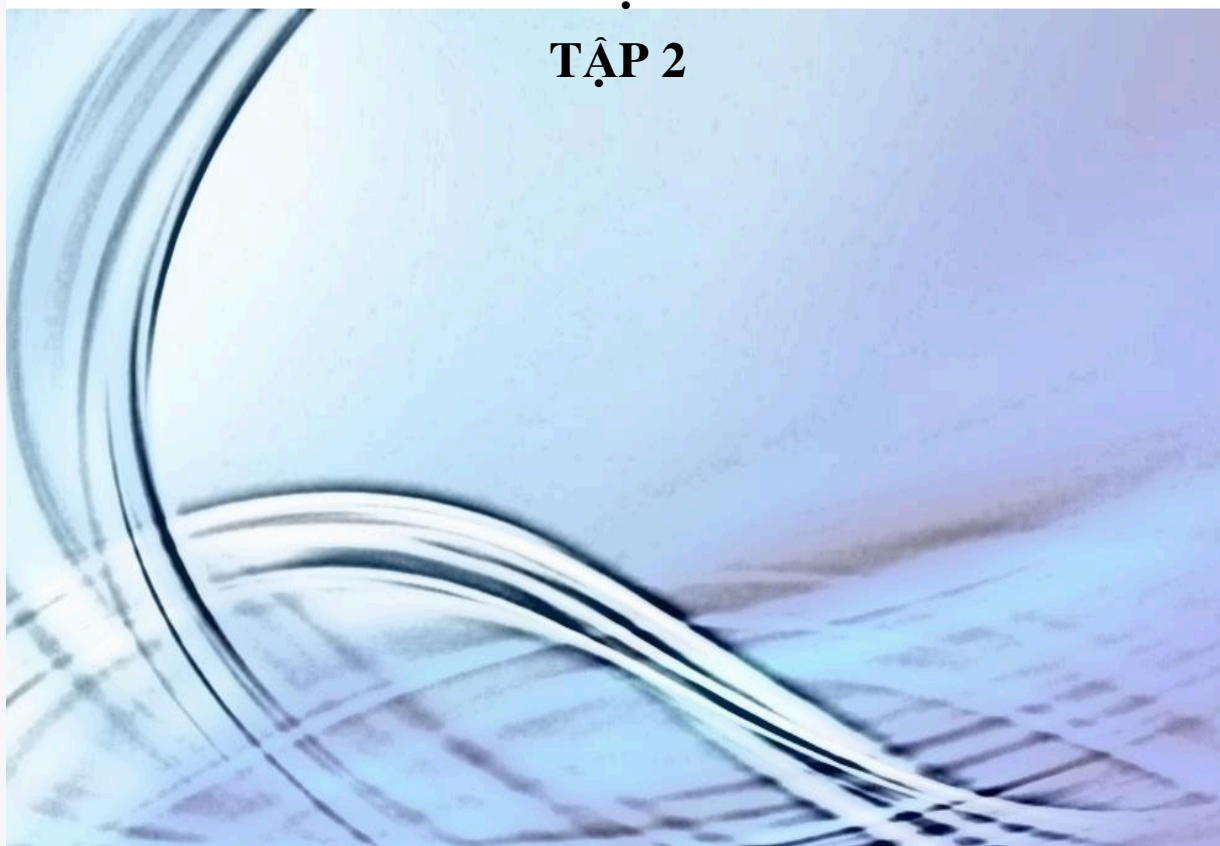


Th.S Nguyễn Quang Thuận



TÀI LIỆU
BỒI DƯỠNG THUẬT TOÁN CƠ BẢN
TẬP 2

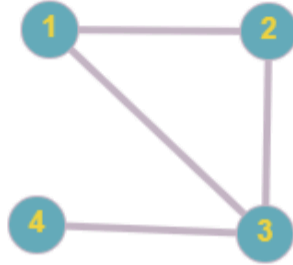


PHẦN 7: ĐỒ THỊ

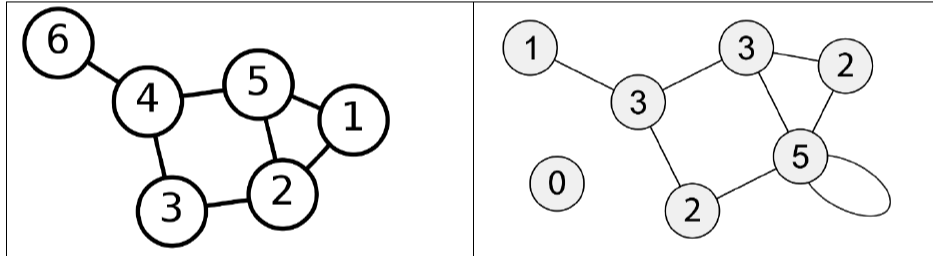
7.1. Khái niệm cơ bản

Đồ thị (Graph) là một cấu trúc toán học dùng để biểu diễn mối quan hệ giữa các cặp đối tượng, chúng ta có thể hình dung nó gồm 2 thành phần:

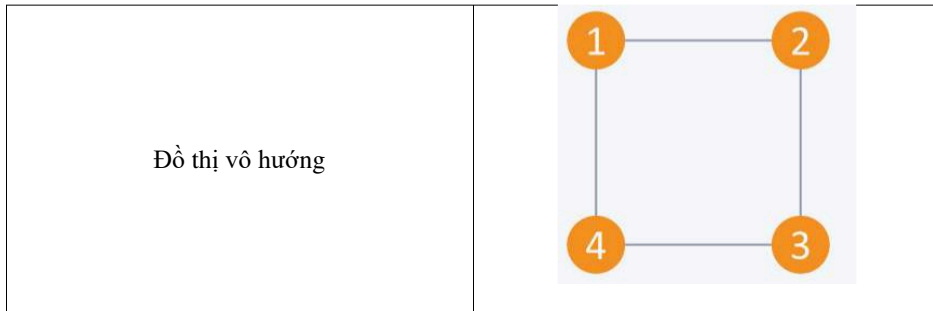
- Nút (Nodes): biểu diễn một đối tượng
- Cạnh (Edges): biểu diễn quan hệ giữa các đối tượng



- Chu trình (Cycle): là đường đi xuất phát từ 1 đỉnh đi qua hết các đỉnh và trở về lại đỉnh ban đầu.
- khuyên là cạnh có cùng điểm xuất phát và điểm đến.



7.2. Phân loại đồ thị



Đồ thị có hướng	
Đồ thị có trọng số	
• CÂY: Là đồ thị vô hướng, trong đó giữa hai nút bất kì luôn tồn tại 1 đường đi duy nhất. • Trong cây, trừ nút gốc ra thì các nút còn lại chỉ có 1 nút cha duy nhất. • Cây không chứa chu trình hay khuyên. • Cây luôn có N đỉnh và N-1 cạnh	Tree

7.3. Biểu diễn đồ thị

7.3.1. Ma trận kề

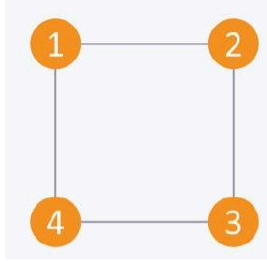
Xét đồ thị $G=(V,E)$ với $V=\{x_1, \dots, x_n\}$

Biểu diễn G bằng ma trận $A=(a_{ij})$, $i,j=1..n$

- $a_{ij}=1$, nếu $\exists (x_i, x_j) \in E$
- $a_{ij}=0$, nếu $\nexists (x_i, x_j) \in E$

7.3.1.1 Biểu diễn đồ thị vô hướng

7.3.1.1.1 Hình minh họa

<pre> i/j : 1 2 3 4 1 : 0 1 0 1 2 : 1 0 1 0 3 : 0 1 0 1 4 : 1 0 1 0 </pre>	
--	--

7.3.1.1.2 mtkvohuong – Ma trận kề vô hướng

Hãy biểu diễn đồ thị đã cho theo dạng ma trận kề.

INPUT

Dữ liệu vào từ file **mtkvohuong.inp** gồm:

- Dòng đầu tiên chứa hai số nguyên **n, m**
- **n** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u, v** tương ứng là có cung nối giữa **u** và **v**.

OUTPUT

- Ghi ra file **mtkcohuong.out** ma trận kề biểu diễn cho đồ thị.

Ràng buộc: $1 \leq n, m \leq 100$

Ví dụ

mtkvohuong.inp	mtkvohuong.out
<pre> 4 4 1 2 1 3 1 4 2 3 </pre>	<pre> 0 1 1 1 1 0 1 0 1 1 0 0 1 0 0 0 </pre>

Code tham khảo

```

#include <bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define maxpt 100
using namespace std;
int main()
{
    freopen("mtkvohuong.inp", "r", stdin);
    freopen("mtkcohuong.out", "w", stdout);
    //Khai bao bien
    int n,m,u,v;
    bool a[maxpt][maxpt];
    memset(a,0,sizeof(a));
    cin >> n >> m;
    FOR(i,1,m)
    {
        cin >> u >> v;
        a[u][v] = a[v][u] = true;
    }
    //Hien thi ma tran

```

```

FOR(i,1,n)
{
    FOR(j,1,n) cout << a[i][j] << " ";
    cout << '\n';
}
return 0;
}

```

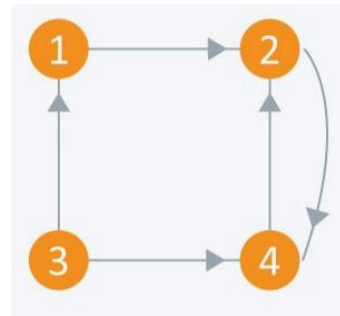
7.3.1.2 Biểu diễn đồ thị có hướng

7.3.1.2.1. Hình minh họa

```

i/j: 1 2 3 4
1: 0 1 0 0
2: 0 0 0 1
3: 1 0 0 1
4: 0 1 0 0

```



7.3.1.2.2. mtkcohuong – Ma trận kề có hướng

Hãy biểu diễn đồ thị đã cho theo dạng ma trận kề.

INPUT

Dữ liệu vào từ file **mtkcohuong.inp** gồm:

- Dòng đầu tiên chứa hai số nguyên **n, m**
- **n** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u, v** tương ứng là có cung nối một chiều giữa **u** và **v**.

OUTPUT

- Ghi ra file **mtkcohuong.out** ma trận kề biểu diễn cho đồ thị.

Ràng buộc: $1 \leq n, m \leq 100$

Ví dụ

mtkcohuong.inp	mtkcohuong.out
4 4	0 1 1 1
1 2	0 0 1 0
1 3	0 0 0 0
1 4	0 0 0 0
2 3	

Code tham khảo

```

#include <bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define maxpt 102
using namespace std;
int main()
{

```

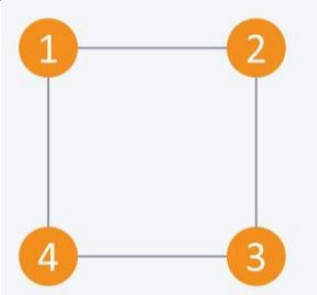
```
freopen("mtkcohuong.inp","r",stdin);
freopen("mtkcohuong.out","w",stdout);
//Khai bao bien
int n,m,u,v;
bool a[maxpt][maxpt];
memset(a,0,sizeof(a));
cin >> n >> m;
FOR(i,1,m)
{
    cin >> u >> v;
    a[u][v] = true;
}
//Hien thi ma tran
FOR(i,1,n)
{
    FOR(j,1,n) cout << a[i][j] << " ";
    cout << '\n';
}
return 0;
}
```

7.3.2. Danh sách kề

Với mỗi đỉnh i, ta lưu một danh sách các nút kề với i

7.3.2.1. Biểu diễn đồ thị vô hướng

7.3.2.1.1 Hình minh họa

Danh sách kề tương ứng với các đỉnh là: 1: 2 4 2: 1 3 3: 2 4 4: 3 1	
---	--

7.3.2.1.2 dskvohuong – Danh sách kề vô hướng

Hãy biểu diễn đồ thị đã cho theo dạng danh sách kề.

INPUT

Dữ liệu vào từ file **dskvohuong.inp** gồm:

- Dòng đầu tiên chứa hai số nguyên **n, m**
- **n** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u,v** tương ứng là có cung nối giữa **u** và **v**.

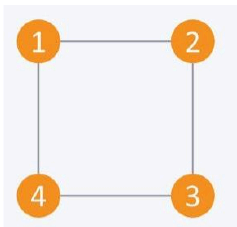
OUTPUT

- Ghi ra file **dskvohuong.out** danh sách kề biểu diễn cho đồ thị.

Ràng buộc: $1 \leq n, m \leq 100$

Ví dụ

dskvohuong.inp	dskvohuong.out	Hình minh họa
----------------	----------------	---------------

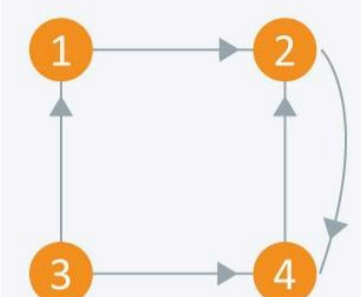
4 4	1: 2 4	
1 2	2: 1 3	
2 3	3: 2 4	
3 4	4: 1 3	
1 4		

Code tham khảo

```
#include<bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define N 102
using namespace std;
int main()
{
    freopen("dskvohuong.inp","r",stdin);
    freopen("dskvohuong.out","w",stdout);
    // KHAI BAO DANH SACH KE
    vector<int> adj[N];
    int n,m,u,v;
    //DOC MA TRAN TU FILE INPUT
    cin >> n >> m;
    FOR(i,1,m)
    {
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    //HIEN THI DANH SACH KE
    FOR(i,1,n)
    {
        if (adj[i].size()>0){
            cout << i << ": ";
            sort(adj[i].begin(),adj[i].end());
            FOR(j,0,adj[i].size()-1) cout << adj[i][j] << " ";
            cout << '\n';
        }
    }
    return 0;
}
```

7.3.2.2. Biểu diễn đồ thị có hướng

7.3.2.2.1 Hình minh họa

Danh sách kề tương ứng với các đỉnh là: 1: 2 2: 4 3: 1 4 4: 2	
---	--

7.3.2.2.1 dskcohuong – Danh sách kề có hướng

Hãy biểu diễn đồ thị đã cho theo dạng danh sách kề.

INPUT

Dữ liệu vào từ file **dskcohuong.inp** gồm:

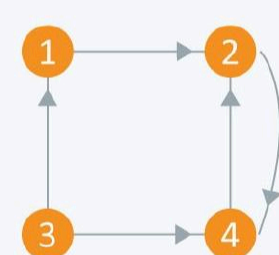
- Dòng đầu tiên chứa hai số nguyên **n, m**
- **n** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u,v** tương ứng là có cung nối một chiều từ **u** đến **v**.

OUTPUT

- Ghi ra file **dskcohuong.out** danh sách kề biểu diễn cho đồ thị.

Ràng buộc: $1 \leq n, m \leq 100$

Ví dụ

dskvohuong.inp	dskvohuong.out	Hình minh họa
4 5 1 2 2 4 3 1 3 4 4 2	1: 2 2: 4 3: 1 4 4: 2	

Code tham khảo

```
#include<bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define N 102
using namespace std;
int main()
{
    freopen("dskcohuong.inp","r",stdin);
    freopen("dskcohuong.out","w",stdout);
    // KHAI BAO DANH SACH KE
    vector<int> adj[N];
    int n,m,u,v;
    //DOC MA TRAN TU FILE INPUT
    cin >> n >> m;
```

```

FOR(i,1,m)
{
    cin >> u >> v;
    adj[u].push_back(v);
    //adj[v].push_back(u);
}
//HIEN THI DANH SACH KE
FOR(i,1,n)
{
    if (adj[i].size()>0){
        cout << i << ": ";
        sort(adj[i].begin(),adj[i].end());
        FOR(j,0,adj[i].size()-1) cout << adj[i][j] << " ";
        cout << '\n';
    }
}
return 0;
}

```

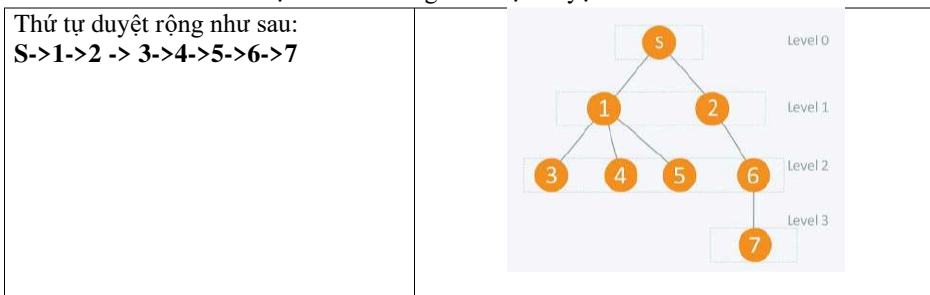
7.4. Phương pháp duyệt đồ thị

- Khi bài toán chúng ta quy về đồ thị thì việc duyệt từng nút trên đồ thị là một việc làm hiển nhiên phải thực hiện để thao tác trên từng nút và giải quyết một bài toán cụ thể nào đó.
- Có thể chia duyệt đồ thị thành 2 cách chính tùy theo thứ tự nút tiếp theo cần duyệt của chúng: duyệt rộng và duyệt sâu.

7.4.1. Duyệt rộng

7.4.1.1. Khái niệm

- Ưu tiên các nút lân cận các nút đang xét được duyệt trước.



Độ phức tạp khi lưu trữ theo danh sách kề : $O(V+E)$

7.4.1.2. Bài tập vận dụng

7.4.1.2.1. bfs – Duyệt rộng

Cho đồ thị có **N** đỉnh và **M** cạnh, nhiệm vụ của bạn là đưa ra thứ tự duyệt rộng của đồ thị với đỉnh xuất phát là đỉnh 1.

INPUT: dữ liệu vào từ file bfs.inp

Dòng đầu tiên là hai số N, M

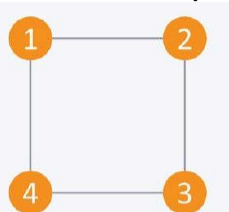
M dòng tiếp theo, mỗi dòng chứa hai số nguyên u,v tương ứng có cung nối giữa hai đỉnh u,v.

OUTPUT: ghi ra file bfs.out thứ tự duyệt rộng xuất phát từ đỉnh 1

RÀNG BUỘC

- $1 \leq N, M \leq 100$
- $1 \leq u, v \leq 100$

VÍ DỤ

bfs.inp	bfs.out	Hình minh họa
4 4 1 2 1 4 2 3 3 4	1 2 4 3	

Code tham khảo

```
#include<bits/stdc++.h>
#define FOR(i,a,b) for (int i=(a);i<=(b);i++)
#define N 102
using namespace std;
/* Function to do BFS of graph
*   s   : start node
*   adj[]: array of vectors
*   vis[]: array to keep track of visited nodes
*/
void bfs(int s, vector<int> adj[], bool d[])
{
    cout << "1 ";
    int u,v;
    queue<int> qu;
    //day nut s vao queue
    qu.push(s);
    //danh dau nut s da duoc tham
    d[s] = true;
    while(qu.size())
    {
        //lay mot dinh tu queue ra
        u = qu.front();
        //xoa dinh do khoi queue
        qu.pop();
        //duyet tat ca ca lan can cua u, day vao queue
        for(int i=0;i<adj[u].size();i++)
        {
            v=adj[u][i];
            if (d[v]==false)
            {
                cout << v << " ";
                d[v] = true;
                qu.push(v);
            }
        }
        cout << '\n';
    }
}
int main()
{
    freopen("bfs.inp","r",stdin);
    freopen("bfs.out","w",stdout);

    int n,m,u,v;
```

```

vector<int> adj[N];
bool d[N];
memset(d,0,sizeof(d));
//doc do thi, luu theo danh sach ke
cin >> n >> m;
FOR(i,1,m)
{
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
bfs(1,adj,d);

return 0;
}

```

7.4.1.2.2. levelnode

Cho một cây gồm **N** nút và **N-1** cung, các nút được đánh số từ 1 đến **N**. Nút gốc nằm ở mức 1. Nhiệm vụ của bạn là đếm số lượng nút ở mức **x**.

INPUT: dữ liệu vào từ file **levelnode.inp** gồm

- Dòng đầu tiên chứa số nguyên **N** là số lượng nút
- **N-1** dòng tiếp theo chứa hai số nguyên **u,v** tương ứng là có cung nối từ **u** đến **v**.
- Dòng tiếp theo chứa số nguyên **x**

OUTPUT: ghi ra file **levelnode.out** số lượng nút ở mức **x**.

RÀNG BUỘC

- $1 \leq N \leq 10^5$
- $1 \leq a, b \leq N$

VÍ DỤ

levelnode.inp	levelnode.out
20 11 1 1 2 13 3 15 4 17 5 11 6 2 7 1 8 15 9 4 10 15 12 5 13 2 14 17 15 15 16 11 17 15 18 9 19 16 20	3

Code tham khảo

```

#include<bits/stdc++.h>
#define FOR(i,a,b) for (int i=(a);i<=(b);i++)
#define N 100001
using namespace std;
void bfs(int s, vector<int> adj[], int level[],int x)
{
    //dem ket qua
    int dem =0;
    int u,v;
    queue<int> qu;
    //day nut s vao queue
    qu.push(s);
    //danh dau nut s da duoc tham
    level[s] = 1;
    while(!qu.empty())
    {
        //lay mot dinh tu queue ra
        u = qu.front();
        //xoa dinh do khoi queue
        qu.pop();
        //duyet tat ca ca lan can cua u, day vao queue
        FOR(i,0,adj[u].size()-1)
        {
            v=adj[u][i];
            if (level[v]==-1)
            {
                //cout << v << " ";
                level[v] = level[u]+1;
                if (level[v]==x) dem++;
                qu.push(v);
            }
        }
    }
    if (x==0) cout << 1;
    else cout << dem;
}

int main()
{
    freopen("levelnode.inp","r",stdin);
    freopen("levelnode.out","w",stdout);
    int e,u,v,x,n;
    vector<int> adj[N];
    int level[N];
    memset(level,-1,sizeof(level));
    //doc do thi, luu theo danh sach ke
    cin >> n;
    FOR(i,1,n-1)
    {
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    cin >> x;
    bfs(1,adj,level,x);
    return 0;
}

```

7.4.1.2.3. Bisland

Cho **N** hòn đảo (đánh số từ 1 đến **N**) được nối với nhau bởi **M** cây cầu. John là một người rất yêu thể thao nhưng lại ngại qua cầu. Bạn hãy giúp John tìm đường đi từ đảo thứ 1 đến đảo thứ **N** sao cho đi qua ít cây cầu nhất.

INPUT

Dữ liệu vào từ file **BISLAND.INP**

Dòng đầu tiên chứa số nguyên **T**

Với mỗi test case

Dòng đầu tiên chứa hai số nguyên **N, M**

M dòng tiếp theo mỗi dòng chứa hai số nguyên **X, Y**. Kí hiệu rằng có cây cầu nối giữa hai đảo **X** và **Y**.

OUTPUT

Với mỗi test case ghi ra file **BISLAND.OUT** một số nguyên là số lượng cây cầu tối thiểu trên đường đi từ đảo 1 đến đảo **N**, mỗi kết quả trên 1 dòng.

RÀNG BUỘC

- $1 \leq T \leq 10$
- $1 \leq N \leq 10^4$
- $1 \leq M \leq 10^5$
- $1 \leq X, Y \leq N$

VÍ DỤ

BISLAND.INP	BISLAND.OUT
2	2
3 2	2
1 2	
2 3	
4 4	
1 2	
2 3	
3 4	
4 2	

Code tham khảo

```
#include <bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define N 10001
#define M 100001
#define T 11
using namespace std;
int bfs(int s,vector<int> adj[], bool vis[],int n)
{
    queue<int> que;
    int level[M];
    int u,v;
    //Day phan tu dau tien vao queue
    que.push(s);
    level[s] = 0;
    vis[s]= true;
    while (!que.empty())
    {
        //lay dinh dau tien trong queue ra
```

```

        u=que.front();
        que.pop();
        //duyet lan can cua u
        FOR(i,0,adj[u].size()-1)
        {
            v=adj[u][i];
            if (vis[v]==false)
            {
                level[v] = level[u]+1;
                if (v==n) return level[n];
                que.push(v);
                vis[v]= true;
            }
        }
    }
    return level[n];
}
int main()
{
    freopen("BISLAND.INP","r",stdin);
    freopen("BISLAND.OUT","w",stdout);

    int t,n,m,x,y;
    bool vis[N];
    memset(vis,0,sizeof(vis));
    cin >> t;
    vector<int> adj[M];
    FOR(k,1,t)
    {
        //nhap du lieu
        cin >> n >> m;
        FOR(i,1,m)
        {
            cin >> x>> y;
            adj[x].push_back(y);
            adj[y].push_back(x);
        }
        cout << bfs(1,adj,vis,n) << endl;
        //reset adj
        FOR(j,0,n-1) adj[j].clear();
        memset(vis,0,sizeof(vis));
    }

    return 0;
}

```

7.4.1.2.4. obattle

Bạn được cho một bản đồ quân sự có **N** hàng và **M** cột. Tại hàng **i**, cột **j** chứa một giá trị **(i,j)** chứa giá trị 0 hoặc 1 tương ứng là không có quân địch hoặc có quân địch. Quân của bạn chỉ còn 1 quả đại bác duy nhất nên bạn cần tìm cách tiêu diệt được số quân địch lớn nhất. Biết rằng đại bác bắn đi sẽ tiêu diệt được 1 nhóm quân có liên thông với nhau. Hai quân lính được xem là liên kề nếu nó liên kề với 8 ô quanh nó.

(x-1,y-1)	(x-1,y)	(x-1,y+1)
(x,y-1)	(x,y)	(x,y+1)
(x+1,y-1)	(x+1,y)	(x+1,y+1)

INPUT

Dữ liệu vào từ file **OBATTLE.INP**

Dòng đầu tiên chứa số nguyên **T** là số bộ testcase

Với mỗi bộ test case

- Dòng đầu tiên chứa hai số nguyên **N** và **M**
- Tiếp theo là ma trận có N hàng và M cột chứa giá trị 0 hoặc 1 là mô tả cho bản đồ.

OUTPUT

Với mỗi bộ test, ghi ra file **OBATTLE.OUT** 2 số nguyên **X** và **Y** tương ứng là số nhóm quân đội và số lượng quân nhiều nhất có thể chết khi bắn quả đại bác cuối cùng.

RÀNG BUỘC

- $1 \leq T \leq 10$
- $1 \leq N, M \leq 1000$

VÍ DỤ

OBATTLE.INP	OBATTLE.OUT
2	2 4
4 6	2 5
0 0 0 1 1 0	
1 1 0 0 0 0	
0 1 0 0 0 0	
0 0 1 0 0 0	
6 4	
1 1 1 1	
0 0 0 0	
0 1 0 0	
1 0 1 0	
1 0 0 0	
1 0 0 0	

Code tham khảo

```
#include<bits/stdc++.h>
using namespace std;
int n,m;
int mat[1002][1002];
int visited[1002][1002];
// counts the number of soldiers in a troop and marks each troop visited in
the visited matrix
int findans(int x, int y)
{
    visited[x][y] = 1;
    // counts the number of soldiers in a troop
```

```

int nodecount = 1;
queue<pair<int,int> > q;
q.push(make_pair(x,y));
while(!q.empty())
{
    x = q.front().first;
    y = q.front().second;
    q.pop();
    for(int i = -1 ; i<=1; i++ )
        for(int j = -1; j<=1; j++)
            if(!visited[x+i][y+j] && mat[x+i][y+j])
            {
                nodecount++;
                q.push(make_pair(x+i,y+j));
                visited[x+i][y+j] = 1;
            }
}
return nodecount;
}
int main()
{
    freopen("OBATTLE.INP", "r", stdin);
    freopen("OBATTLE.out", "w", stdout);
    int t;
    scanf("%d",&t);
    while(t--)
    {
        scanf("%d%d",&n,&m);
        memset(mat,0,sizeof(mat));
        memset(visited,0,sizeof(visited));
        for(int i = 1; i<=n ; i++)
            for(int j = 1; j<= m; j++)
                scanf("%d",&mat[i][j]);
        // maintains the maximum number of soldiers in any troop
        int ans = 0;
        // counts the total number of troops
        int counter = 0;
        for(int i = 1; i <= n ; i++)
            for(int j = 1; j<= m; j++)
            {
                // it finds the first 1 in the matrix representing an uncounted troop
                if(!visited[i][j] && mat[i][j])
                {
                    ans = max( findans(i,j), ans );
                    counter++;
                }
            }
        cout<<counter<<" "<<ans<<endl;
    }
    return 0;
}

```

7.4.1.2.5. HPARTY

H là một cô gái rất xinh đẹp, trong một buổi tiệc rất nhiều chàng trai muốn nhảy với H, danh sách chàng trai được đánh số từ 1 đến N. Cho biết danh sách những đôi được nhảy với nhau, bạn hãy xác định độ may mắn của các chàng trai. Biết rằng độ may mắn được tính dựa vào việc chàng trai có được trực tiếp nhảy với H hay không? Nếu nhảy trực tiếp độ may mắn là 1, nếu không được nhảy trực tiếp nhưng lại nhảy với người được nhảy với H sẽ có độ may mắn là 2, tương tự như vậy với các độ may mắn khác. Nếu chàng trai không

được nhảy với H cũng như tập những người may mắn được nhảy với H sẽ xem như không may mắn và nhận giá trị “-1”.

Lưu ý: H được đánh số là 0

INPUT

- Dữ liệu vào từ file **HPARTY.INP**
- Dòng đầu tiên chứa 2 số nguyên **N** và **M** tương ứng là số người trong bữa tiệc và số lượng đôi nhảy với nhau.
- **M** dòng tiếp theo, mỗi dòng là hai giá trị **x,y** tương ứng là hai người **x,y** đã nhảy với nhau.

OUTPUT

- Ghi ra file **HPARTY.OUT** **N-1** dòng, tại dòng thứ **i** là độ may mắn của người thứ **i**.

RÀNG BUỘC

- $N \leq 1000$
- $M \leq (N*(N-1))/2$

VÍ DỤ

HPARTY.INP	HPARTY.OUT
5 6	1
0 1	1
0 2	2
3 2	2
2 4	
4 3	
1 2	

Code tham khảo

```
#include <cstdio>
#include <cstdlib>
#include <queue>
#include <iostream>
using namespace std;
int main() {
    freopen("HPARTY.INP", "r", stdin);
    freopen("HPARTY.out", "w", stdout);
    int numPerson, numDance;
    int i, j;
    cin >> numPerson >> numDance;
    int gnum[numPerson];
    bool adj[numPerson][numPerson];
    for (i = 0; i < numPerson; i++)
    {
        for (j = 0; j < numPerson; j++)
        {
            adj[i][j] = false;
        }
        gnum[i] = -1;
    }
    for (i = 0; i < numDance; i++) {
        int a, b;
        cin >> a >> b;
        adj[a][b] = adj[b][a] = true;
    }
}
```

```

//BFS
gnum[0] = 0;
queue<int> q;
q.push(0);
while (!q.empty())
{
    int thisPerson = q.front();
    q.pop();
    for (i = 1; i < numPerson; i++)
    {
        if (adj[i][thisPerson] && gnum[i] == -1)
        {
            gnum[i] = gnum[thisPerson]+1;
            q.push(i);
        }
    }
}

for (i = 1; i < numPerson; i++) {
    cout<<gnum[i]<<endl;
}
return 0;
}

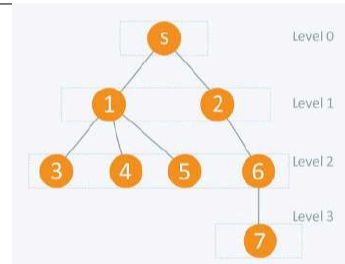
```

7.4.2. Duyệt sâu

7.4.2.1. Khái niệm

Ưu tiên nút con của nút đang xét để duyệt trước.

Thứ tự duyệt rộng như sau:
S->1->3 -> 4->5->2->6->7



Độ phức tạp nếu lưu đồ thị theo danh sách kề: **$O(V + E)$**

7.4.2.2. Bài tập vận dụng

7.4.2.2.1.dfs – duyệt sâu

Cho đồ thị có **N** đỉnh và **M** cạnh, nhiệm vụ của bạn là đưa ra thứ tự duyệt sâu của đồ thị với đỉnh xuất phát là đỉnh 1.

INPUT: dữ liệu vào từ file **dfs.inp**

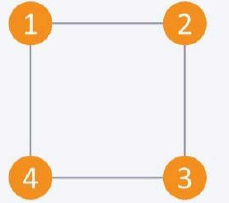
- Dòng đầu tiên là hai số **N, M**
- **M** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u, v** tương ứng có cung nối giữa hai đỉnh **u, v**.

OUTPUT: ghi ra file **dfs.out** thứ tự duyệt rộng xuất phát từ đỉnh 1

RÀNG BUỘC

- $1 \leq N, M \leq 100$
- $1 \leq u, v \leq 100$

VÍ DỤ

dfs.inp	dfs.out	Hình minh họa
4 4 1 2 1 4 2 3 3 4	1 2 3 4	

Code tham khảo

```
#include<bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define N 100001
using namespace std;
int n,m,x,dem=1;
vector<int> adj[N];
bool vis[N];
int DFS(int x)
{
    cout << x << " ";
    //danh dau x da duoc tham
    vis[x]= true;
    //duyet tung lan can cua x
    for(int i=0;i < adj[x].size();i++)
    //FOR(i,0,adj[x].size()-1)
    {
        if(!vis[adj[x][i]])
        {
            DFS(adj[x][i]);
        }
    }
    return dem;
}
int main()
{
    freopen("dfs.inp","r",stdin);
    freopen("dfs.out","w",stdout);

    int u,v;
    cin >> n >> m;
    FOR(i,1,m)
    {
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    memset(vis,0,sizeof(vis));
    DFS(1);
    return 0;
}
```

7.4.2.2.2. unreachable – Nút không thể đến được

Cho đồ thị có **N** đỉnh, **M** cạnh và nút **x**. Nhiệm vụ của bạn là đếm số lượng nút không liên thông với nút **x**.

INPUT: dữ liệu vào từ file **unreachable.inp**

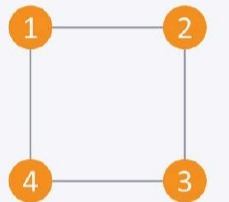
- Dòng đầu tiên là hai số **N, M**
- **M** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u,v** tương ứng có cung nối giữa hai đỉnh **u,v**.
- Dòng cuối cùng là số nguyên **x**

OUTPUT: ghi ra file **unreachable.out** số lượng nút không liên thông với **x**.

RÀNG BUỘC

- $1 \leq N, M \leq 10^5$
- $1 \leq x \leq N$

VÍ DỤ

unreachable.inp	unreachable.out	Hình minh họa
4 4 1 2 1 4 2 3 3 4	0	

Code tham khảo

Cách 1: Sử dụng đệ quy

```
#include<bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define N 100001
using namespace std;
int n,m,x,dem=1;
vector<int> adj[N];
bool vis[N];
int DFS(int x)
{
    //danh dau x da duoc tham
    vis[x]= true;
    //duyet tung lan can cua x
    for(int i=0;i < adj[x].size();i++)
    //FOR(i,0,adj[x].size()-1)
    {
        if(!vis[adj[x][i]])
        {
            dem++;
            DFS(adj[x][i]);
        }
    }
    return dem;
}
int main()
{
    freopen("unreachable.inp","r",stdin);
    freopen("unreachable.out","w",stdout);

    int u,v;
    cin >> n >> m;
    FOR(i,1,m)
    {
        cin >> u >> v;
        adj[u].push_back(v);
    }
}
```

```

        adj[v].push_back(u);
    }
    cin >> x;
    //khởi tạo vis
    memset(vis,0,sizeof(vis));
    cout << n-DFS(x);
    return 0;
}

```

Cách 2: Sử dụng vòng lặp

```

#include<bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define N 100001
using namespace std;
int n,m,x;
vector<int> adj[N];
int DFS(int x)
{
    // khai báo
    stack<int> mystack;
    bool vis[N];
    int u,v,dem=1;
    //khởi tạo
    memset(vis,0,sizeof(vis));
    // day phan tu can duyet vao stack
    mystack.push(x);
    vis[x]=true;
    // thuc hien kiem tra stack
    while (!mystack.empty())
    {
        //lay phan tu trong stack
        u = mystack.top();
        mystack.pop();
        // duyet tung lan can cua u
        for (int i=0;i<adj[u].size();i++)
        //FOR(i,0,adj[u].size()-1)
        {
            v=adj[u][i];
            if (!vis[v])
            {
                dem++;
                // day v vao stack
                mystack.push(v);
                //danh dau v da duoc tham
                vis[v] = true;
            }
        }
    }
    return dem;
}
int main()
{
    freopen("unreachable.inp","r",stdin);
    freopen("unreachable.out","w",stdout);

    int u,v;
    cin >> n >> m;
    FOR(i,1,m)
    {
        cin >> u >> v;
        adj[u].push_back(v);
    }
}

```

```

adj[v].push_back(u);
}
cin >> x;
cout << n-DFS(x);
return 0;
}

```

7.4.2.2.3.TFRIENDS - THÀNH VÀ BẠN GÁI

Thành là một thanh niên cao to và đẹp trai, với vẻ ngoài điển trai đã khiến rất nhiều cô gái phải chết mê chết mệt. Cho biết bản đồ khu vực của Thành đang sống gồm **N** thành phố được nối với nhau bởi **N-1** tuyến đường. Thành đang ở thành phố có số thứ tự 1, có thể xem bản đồ như dạng cây với thành phố Thành đang sống là gốc. Có **Q** cô gái say đắm trước vẻ nam tính của Thành. Thành là người ngại vận động nên anh muốn tìm người bạn gái ở gần mình nhất, khoảng cách giữa hai thành phố chính là số đoạn đường phải đi qua giữa hai thành phố. Bạn hãy giúp Thành tìm ra số thứ tự của thành phố mà cô gái thỏa mãn điều kiện đang sống.

INPUT

Dữ liệu vào từ file **TFRIENDS.INP**

- Dòng đầu tiên chứa số nguyên **N** là số thành phố
- **N-1** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u,v** mô tả tồn tại đường đi giữa hai thành phố **u** và **v**
- Dòng tiếp theo chứa số nguyên **Q** là số cô gái
- **Q** dòng tiếp theo, mỗi dòng chứa một số nguyên là số thứ tự của thành phố mà cô gái thứ **i** đang sống.

OUTPUT

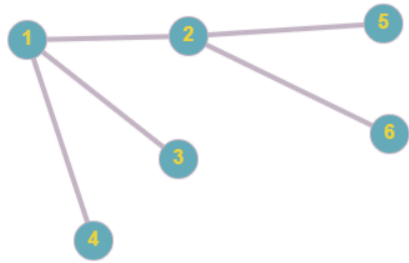
Ghi ra file **TFRIENDS.OUT** một số nguyên duy nhất là số thứ tự của thành phố mà cô gái thỏa mãn đang sống. Nếu có nhiều cô gái cùng thỏa mãn điều kiện về khoảng cách thì sẽ ưu tiên cô gái có số thứ tự thành phố nhỏ hơn.

Biết rằng không có cô gái nào sống ở thành phố 1 và không có hai cô gái sống chung một thành phố.

RÀNG BUỘC

- $2 \leq N \leq 1000$
- $1 \leq u, v \leq N$
- $1 \leq Q \leq (N-1)$

VÍ DỤ

TFRIENDS.INP	TFRIENDS.OUT	Hình minh họa
6 1 2 1 3 1 4 2 5 2 6 4 5 6 3 4	3	

Code tham khảo

```
#include <bits/stdc++.h>
using namespace std;
vector<int> arr[1001];
int visited[1001]={0};

void dfs(int s,int length)
{
    visited[s]=length;

    for(int i=0;i<arr[s].size();i++)
    {
        if(visited[arr[s][i]]==0)
        {
            dfs(arr[s][i],length+1);
        }
    }
}

int main()
{
    freopen("TFRIENDS.INP", "r", stdin);
    freopen("TFRIENDS.out", "w", stdout);
    int n;
    cin>>n;

    for(int i=0;i<n-1;i++)
    {
        int x,y;
        cin>>x>>y;

        arr[x].push_back(y);
        arr[y].push_back(x);
    }

    int length=1;

    dfs(1,length);

    int q;
    cin>>q;

    int minIndex=INT_MAX;
    int minNum=0;
```

```

while(q-->0){
    int num;
    cin >> num;
    if(minIndex > visited[num]){

        minIndex = visited[num];
        minNum=num;
    }
}
cout << minNum << endl;

return 0;
}

```

7.4.2.2.4. HVERTEX – ĐỈNH HẠNH PHÚC

Cho một cây có N đỉnh và M cạnh, đỉnh được gọi là hạnh phúc nếu nó có số cây con lớn hơn nút cha. Nhiệm vụ của bạn là đếm xem trên cây có bao nhiêu đỉnh hạnh phúc.

INPUT

Dữ liệu vào từ file **HVERTEX.INP**

- Dòng đầu tiên chứa 2 số nguyên N và M
- M dòng tiếp theo, mỗi dòng chứa hai số nguyên X và Y mô tả có cạnh nối giữa hai đỉnh X và Y .

OUTPUT

Ghi ra file **HVERTEX.OUT** một số nguyên duy nhất là số đỉnh hạnh phúc.

RÀNG BUỘC

- $1 \leq N \leq 100000$
- $0 \leq M \leq N-1$
- $1 \leq X, Y \leq N$

VÍ DỤ

HVERTEX.INP	HVERTEX.OUT	Hình minh họa
<pre> 4 3 1 2 2 3 2 4 </pre>	1	

Code tham khảo

```

#include<bits/stdc++.h>
#define maxpt 100010
using namespace std;

bool visited[maxpt];
vector < vector < int > > v(maxpt);
vector < pair < int, int > > pc(maxpt);

void dfs(int node, int parent)

```

```

{
    int childs = 0;
    visited[node] = true;
    for(int i = 0; i < v[node].size(); i++)
    {
        if(!visited[v[node][i]])
        {
            dfs(v[node][i], node);
            childs++;
        }
    }
    pc[node] = make_pair(parent, childs);
}

int main()
{
    freopen("HVERTEX.INP", "r", stdin);
    freopen("HVERTEX.out", "w", stdout);
    pc[0] = make_pair(0, INT_MAX);
    memset(visited, false, sizeof(visited));
    int nodes, edges, node_count = 0;
    cin >> nodes >> edges;
    for(int i = 0; i < edges; i++)
    {
        int a, b;
        cin >> a >> b;
        v[a].push_back(b);
        v[b].push_back(a);
    }
    for(int i = 1; i <= nodes; i++)
    {
        if(!visited[i])
        {
            dfs(i, 0);
        }
    }
    for(int i = 1; i <= nodes; i++)
    {
        if(pc[i].second > pc[pc[i].first].second)
            node_count++;
    }
    cout << node_count;
}

```

7.4.2.2.5. MNODES - Minimum Nodes

Cho một cây với N nút, nút thứ i được gán một giá trị $val[i]$. Cho Q truy vấn với mỗi truy vấn có dạng $X K$. Với mỗi truy vấn, bạn phải tìm đường đi xuất phát từ nút X sao cho tổng các nút trên đường đi phải lớn hơn hoặc bằng K và số lượng nút trên đường đi phải đảm bảo là ít nhất. Nếu tồn tại đường đi thỏa mãn yêu cầu thì in ra số lượng nút trên đường đi, ngược lại in ra “-1”.

INPUT

Dữ liệu vào từ file **MNODES.INP**

- Dòng đầu tiên chứa hai số nguyên N và Q .
- Dòng thứ hai chứa N số nguyên cách nhau bởi khoảng trắng, số nguyên thứ i mô tả cho giá trị $val[i]$.

- **N-1** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u, v** mô tả tồn tại đường đi giữa hai đỉnh **u** và **v**
- **Q** dòng tiếp theo, mỗi dòng chứa một truy vấn, tại dòng thứ **i** mô tả cho cặp **X_i** và **K_i**.

OUTPUT

Với mỗi truy vấn **Q** ghi ra file **MNODES.OUT** trên một dòng là kết quả của truy vấn.

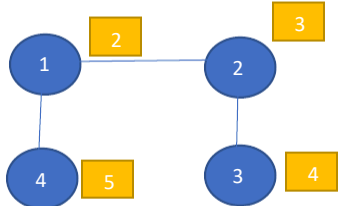
RÀNG BUỘC

- $1 \leq N \leq 10^5$
- $1 \leq Q \leq 10$
- $1 \leq \text{val}[i] \leq 10^9$
- $1 \leq U, V, X \leq N$
- $1 \leq K \leq 10^{10}$

VÍ DỤ

MNODES.INP	MNODES.OUT
4 2	2
2 3 4 5	2
1 2	
2 3	
1 4	
1 6	
2 5	

GIẢI THÍCH

	• Với truy vấn 1 6 Ta thấy rằng có 2 đường đi xuất phát từ 1: 1->2->3 → sum = 9 1->4 → sum=7 Cả hai đường đi đều thỏa mãn sum >=6 nhưng đường đi qua ít đỉnh là 1->4 nên ta chọn đường đi này.
--	---

Code tham khảo

```
#include<bits/stdc++.h>
using namespace std;
const int mx = 1e5+10;

vector<int> gr[mx];
long long val[mx];
int u,v,q,n,x,i,ans1;

void dfs(int node,int cnt,int par,long long sum1,long long k) {
    sum1 += val[node];
    cnt++;
    if( sum1 >= k ){
        ans1 = min(ans1,cnt);
    }
    for(int i=0; i<gr[node].size(); i++) {
```

```

        int nxt=gr[node][i];
        if(nxt == par) {
            continue;
        }
        dfs(nxt,cnt,node,sum1,k);
    }
}

int main() {

    freopen("MNODES.INP", "r", stdin);
    freopen("MNODES.out", "w", stdout);
    long long k;
    scanf("%d %d",&n, &q);
    for(int i=1; i<=n; i++) {
        scanf("%d",&val[i]);
    }
    for(int i=1; i<=n-1; i++) {
        scanf("%d %d",&u, &v);
        gr[u].push_back(v);
        gr[v].push_back(u);
    }
    for(int i=0; i<q; i++) {
        scanf("%d %lld",&x, &k);
        ans1 = INT_MAX;
        dfs(x,0,0,0,k);
        if(ans1 == INT_MAX) {
            ans1=-1;
        }
        printf("%d\n",ans1);
    }
    return 0;
}

```

7.4.2.2.6. SSEE – SIGHTSEEING

Sumo là một đứa trẻ rất thích đi du lịch qua những ngọn núi. Có N điểm trên các ngọn núi, mỗi điểm sẽ có một độ cao là $H[i]$ được nối với nhau bởi M tuyến đường một chiều. Sumo nghĩ rằng một hành trình được cho là đặc biệt nhất nếu $H[\text{điểm kết thúc}] - H[\text{điểm xuất phát}]$ có giá trị lớn nhất. Nhiệm vụ của bạn là giúp Sumo tìm ra giá trị $H[\text{điểm kết thúc}] - H[\text{điểm xuất phát}]$ lớn nhất có thể.

INPUT

Dữ liệu vào từ file **SSEE.INP**

Dòng đầu tiên chứa số nguyên T là số bộ testcase

Với mỗi testcase

- Dòng đầu tiên chứa 2 số N và M
- Dòng thứ hai chứa N số nguyên, số nguyên thứ i biểu diễn cho $H[i]$
- M dòng tiếp theo, mỗi dòng chứa hai số nguyên a_i và b_i .

OUTPUT

Với mỗi test case ghi ra file **SSEE.OUT** trên một dòng, một số nguyên duy nhất là giá trị chênh lệch chiều cao lớn nhất

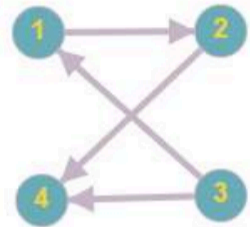
RÀNG BUỘC

- $1 \leq T \leq 10$
- $0 \leq H[i] \leq 10^9$
- $1 \leq a_i, b_i \leq N$

SUBTASK

- Subtask 1: $1 \leq N \leq 1000$; $1 \leq N \leq 2000$
- Subtask 2: $1 \leq N \leq 10^5$; $1 \leq N \leq 2 \cdot 10^5$.

VÍ DỤ

SSEE.INP	SSEE.OUT	Hình minh họa
1 4 5 2 4 10 7 2 4 1 2 3 1 3 4 4 1	5	

Code tham khảo

```
#include <bits/stdc++.h>
using namespace std;
#define FOR(i, a, b) for(int i=(a);i<(b);i++)
#define ALL(V) V.begin(), V.end()
#define SZ(V) (int)V.size()
#define PB push_back
#define MP make_pair

typedef vector<int> VI;
typedef pair<int, int> PII;

const int MAX = 100007;

VI G[MAX];
int H[MAX];
int res;
bool U[MAX];

void dfs(int v, int sh)
{
    res = max(res, H[v] - sh);
    U[v] = 1;
    FOR(i, 0, SZ(G[v]))
    {
        int to = G[v][i];
        if (U[to]) continue;
        dfs(to, sh);
    }
}

int main()
{

```

```

freopen("SSEE.INP", "r", stdin);
freopen("SSEE.OUT" , "w" , stdout);
int t;
cin >> t;
FOR(tt,0,t)
{
    int n , m;
    cin >> n >> m;
    FOR(i,0,n)
    {
        G[i].clear();
        U[i] = 0;
    }
    res = 0;

    FOR(i,0,n)
    {
        scanf("%d" , &H[i]);
    }

    FOR(i,0,m)
    {
        int a , b;
        scanf("%d%d" , &a , &b);
        --a; --b;
        G[a].PB(b);
    }
    vector<PII> A;
    FOR(i,0,n)
    {
        A.PB(MP(H[i] , i));
    }
    sort(ALL(A));

    FOR(i,0,n)
    {
        int v = A[i].second;
        if (U[v]) continue;
        dfs(v , A[i].first);
    }
    cout << res << endl;
}

return 0;
}

```

7.4.2.2.7. MEDGES – SỐ CẠNH LỚN NHẤT

Cho đồ thị vô hướng G có N đỉnh và M cạnh. Nhiệm vụ của bạn là xác định số cạnh tối đa trong bất kì thành phần liên thông nào của đồ thị.

Giả sử như đồ thị có k thành phần liên thông với số cạnh trong mỗi thành phần liên thông là $E_1, E_2, \dots, E_k$ kết quả là $\max(E_1, E_2, \dots, E_k)$

Lưu ý: đồ thị có thể chứa nhiều đường đi giữa hai đỉnh và khuyên.

N đỉnh được đánh số từ 1 đến N .

INPUT

Dữ liệu vào từ file **MEDGES.INP** :

- Dòng đầu tiên là 2 số nguyên **N** và **M**
- **M** dòng tiếp theo, mỗi dòng chứa hai số nguyên **a** và **b** mô tả tồn tại đường đi giữa hai đỉnh **a** và **b**.

OUTPUT

Ghi ra file **MEDGES.OUT** một số nguyên là kết quả của bài toán.

RÀNG BUỘC

- $1 \leq N \leq 10^5$.
- $1 \leq M \leq 10^5$.
- $1 \leq a, b \leq N$.

VÍ DỤ

MEDGES.INP	MEDGES.OUT
6 3 1 2 2 3 4 5	2

GIẢI THÍCH

Đồ thị có 3 thành phần liên thông

- 1,2,3: có 2 cạnh
 - 4,5: có 1 cạnh
 - 6: có 0 cạnh
- ⇒ Thành phần liên thông có 2 cạnh là lớn nhất.

Code tham khảo

```
#include<bits/stdc++.h>
#define maxpt 100005
using namespace std;
vector<int>v[maxpt];
bool vis[maxpt];
int siz,edges;
void dfs(int i)
{
    vis[i]=1;
    //siz++;
    for(int j=0;j<v[i].size();j++)
    {
        edges++;
        if(!vis[v[i][j]])
            dfs(v[i][j]);
    }
}
int main()
{
    freopen("MEDGES.INP", "r", stdin);
    freopen("MEDGES.out", "w", stdout);
    int n,m;
    cin>>n>>m;
    for(int i=0;i<m;i++)
    {
        int a,b;
        cin>>a>>b;
        v[a].push_back(b);
        v[b].push_back(a);
    }
}
```

```

    }
    int maxedges=0;
    for(int i=1;i<=n;i++)
    {
        if(!vis[i])
        {
            dfs(i);
            maxedges=max(maxedges,edges/2);
            siz=0;edges=0;
        }
    }
    cout<<maxedges<<"\n";
    return 0;
}

```

7.5 Đường đi ngắn nhất trên đồ thị

7.5.1. Bài tập cơ bản

7.5.1.1. Phát biểu bài toán

Cho đồ thị vô hướng có trọng số gồm **N** đỉnh và **M** cạnh, nhiệm vụ của bạn là tìm đường đi ngắn nhất từ đỉnh 1 đến tất cả các đỉnh còn lại.

INPUT: dữ liệu vào từ file **spath.inp**

- Dòng đầu tiên chứa hai số nguyên **N,M**
- **M** dòng tiếp theo, mỗi dòng chứa ba số nguyên **u,v** và **w** tương ứng là có cạnh nối giữa hai đỉnh **u, v** và trọng số của cạnh là **w**.

OUTPUT: ghi ra file **spath.out** khoảng cách từ đỉnh 1 đến các đỉnh còn lại, in ra '1000000000' nếu không có đường đi từ đỉnh 1 đến đỉnh đang xét.

RÀNG BUỘC

- $1 \leq N \leq 10^4$
- $1 \leq M \leq 10^6$
- $1 \leq u, v \leq N$
- $1 \leq w \leq 1000$

VÍ DỤ

spath.inp	spath.out	Hình minh họa
5 5 1 2 5 1 3 2 3 4 1 1 4 6 3 5 5	5 2 3 7	

7.5.1.2. Ý tưởng

Về cơ bản ý tưởng của bài toán như sau:

Ta áp dụng phương pháp quy hoạch động để tìm đường đi ngắn nhất

Bài toán tìm đường đi ngắn nhất từ s đến t sẽ quy về bài toán con là tìm đường đi ngắn nhất từ s đến các đỉnh có cạnh nối tới t .

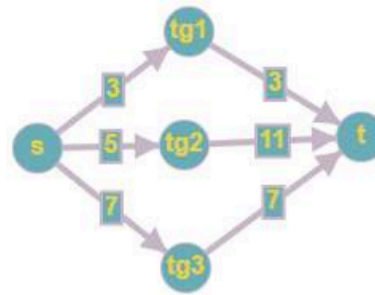
Ví dụ như hình vẽ:

Gọi $d[t]$ lưu đường đi ngắn nhất từ đỉnh s đến đỉnh t

Từ đây ta dễ dàng nhận thấy

$$d[t] = \min(d[t], d[v] + c[v][t])$$

với mọi v là các nút có đường đi đến t .



7.5.1.3. Thuật toán Bellman -Ford

7.5.1.3.1. ý tưởng

B1: Khởi tạo $d[1] = 0$; $d[i] = \text{MAXPT}$ với mọi i từ 2 đến n .

B2: u : duyệt qua tất cả các đỉnh trong đồ thị:

Với mỗi đỉnh u duyệt qua, ta sử dụng u để xét cập nhật tất cả các đỉnh v mà cung (u,v) thuộc đồ thị.

$$d[v] = \min(d[v], d[u] + c[u][v])$$

B3: Lặp lại **B2** cho đến khi không có đỉnh nào cập nhật lại khoảng cách thì kết thúc

Với ý tưởng trên, ta lưu trữ theo **danh sách cạnh** sẽ có độ phức tạp là $O(V.E)$

Lưu trữ theo **ma trận kề** sẽ là $O(V^3)$

Ví dụ về lưu trữ đồ thị theo danh sách cạnh

LƯU TRỮ

SAMPLE INPUT

5	5
1	2 5
1	3 2
3	4 1
1	4 6
3	5 5

adj

Chỉ số						
1	5	2	2	3	6	4
2						
3	4	1	5	5		
4						

N,M: số đỉnh và số cạnh

Biểu diễn bằng danh sách kề, với mỗi phần tử là cặp (trong số, nút kề)

7.5.1.3.2. Code tham khảo

```
#include<bits/stdc++.h>
using namespace std;
```

```

#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define fs first
#define sc second
#define N 10010
#define INF 1000000000
/*const int N = 1E4 + 10;
const int INF = 1E9;*/

typedef pair<int, int> II;
vector<II> adj[N];
int n, m;

int d[N];
bool inqueue[N];

void bellman(int u){
    int i, v, uv;
    queue<int> qu;

    FOR(i,1,n) d[i] = INF;

    d[u]=0;
    qu.push(u); inqueue[u]=true;

    while (qu.size()){
        u=qu.front(); inqueue[u]=false;
        qu.pop();

        for (i=0; v=adj[u][i].sc; i++){
            uv=adj[u][i].fs;
            if (d[v]>d[u]+uv){
                d[v]=d[u]+uv;
                if (!inqueue[v]){
                    qu.push(v);
                    inqueue[v]=true;
                }
            }
        }
    }
}

int main(){
    ios_base::sync_with_stdio(false); cin.tie(0);

    freopen("spath.INP","r",stdin);
    freopen("spath.OUT","w",stdout);

    int p, q, w;
    cin >> n >> m;
    while (m--) {
        cin >> p >> q >> w;
        adj[p].push_back(II(w, q));
    }
    FOR(i,1,n) adj[i].push_back(II(0,0));
    bellman(1);
    FOR(i, 2, n) cout << d[i] << " ";
    return 0;
}

```

7.5.1.3.3. Một số lưu ý

- Không thể áp dụng giải thuật với đồ thị có chu trình âm

- Có thể áp dụng giải thuật với đồ thị có trọng số âm.

7.5.1.4. Thuật toán Dijkstra

7.5.1.4.1. Ý tưởng

- Chiến lược của giải thuật Dijkstra giống với ý tưởng tổng thể để tối ưu hóa từng nút, tuy nhiên thay vì ở bellman-ford việc chọn đỉnh u để tối ưu thì ta phải duyệt toàn bộ các đỉnh, điều này làm tốn khá nhiều thời gian tính toán.
- Để cải thiện điểm này ở Bellman-Ford, chúng ta chọn u là đỉnh nhỏ nhất $d[u]$ chưa được tối ưu.
- Để hiểu chi tiết ta tìm hiểu các bước cụ thể:

Bước 1: Khởi tạo:

$d[s] = 0$ và $d[v] = \infty$ với $\forall v \neq s$

Khởi tạo mảng Free đều False (mảng để cố định nhãn)

$\text{Free}(v) = \begin{cases} \text{True} : \text{đường đi từ } s \text{ đến } v \text{ đã là đường đi ngắn nhất} \\ \text{False} : \text{đường đi từ } s \text{ đến } v \text{ chưa phải là đường đi ngắn nhất} \end{cases}$

Bước 2: Lặp lại các bước cho tới khi $u=f$

- ✓ **2.1. Cố định nhãn:** Chọn trong các đỉnh có nhãn tự do, lấy ra đỉnh u là đỉnh có $d[u]$ nhỏ nhất, và cố định nhãn đỉnh u.
- ✓ **2.2. Sửa nhãn:** Duyệt đỉnh u, xét tất cả những đỉnh v và sửa lại các $d[v]$ theo công thức: $d[v] := \min(d[v], d[u] + c[u, v])$

7.5.1.4.2. Code tham khảo

```
#include <bits/stdc++.h>
using namespace std;

#define FOR(i, a, b) for (int i = (a); i <= (b); i++)
#define fs first
#define sc second

typedef pair <int, int> II;
const int N = 1E4 + 10;
const int INF = 1E9;

vector <II> adj[N];
int n, m;
int d[N];

void Read_Input() {
    freopen("SPATH.INP", "r", stdin);
    freopen("SPATH.OUT", "w", stdout);
    cin >> n >> m;
    FOR(i, 1, m) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back(II(w, v));
        // adj[v].push_back(II(w, u));
    }
}

void Dijkstra() {
    priority_queue <II, vector <II>, greater <II> > heap;
    FOR(i, 2, n) d[i] = INF;
```

```

heap.push(II(d[1], 1));
while (heap.size()) {
    int u = heap.top().sc;
    int du = heap.top().fs;
    heap.pop();
    //truong hop dinh u da duoc cap nhat thi bo qua
    if (du != d[u]) continue;
    //xet tat ca cac lan can cua u
    FOR(i, 0, (int) adj[u].size()-1) {
        int v = adj[u][i].sc;
        //Neu dung u cap nhat duoc cac lan can thi cap nhat thoi
        if (d[v] > d[u] + adj[u][i].fs) {
            d[v] = d[u] + adj[u][i].fs;
            heap.push(II(d[v], v));
        }
    }
}
FOR(i, 2, n) cout << d[i] << " ";
}

int main() {
    Read_Input();
    Dijkstra();
    return 0;
}

```

7.5.1.4.3. Một số lưu ý

- Thuật toán không áp dụng với đồ thị có trọng số âm.
- Độ phức tạp $O(V^2)$ khi lưu trữ đồ thị theo danh sách cạnh.

7.5.2. Bài tập vận dụng

7.5.2.1. MCITY – TÌM ĐƯỜNG ĐI NGẮN NHẤT

Timon đang trong quốc gia có N thành phố (được đánh số từ 1 đến N) được nối với nhau bởi M tuyến đường hai chiều. Tại quốc gia Timon đang sống sắp tổ chức một kì thi Học sinh giỏi, kì thi này chỉ được tổ chức tại K thành phố cho trước. Bài toán đặt ra là cho Q truy vấn, mỗi truy vấn số thứ tự thành phố mà Timon đang sống. Nhiệm vụ của bạn là tìm đường đi ngắn nhất từ thành phố Timon đang sống đến thành phố có tổ chức kì thi gần nhất.

INPUT

Dữ liệu vào từ file **MCITY.INP**

- Dòng đầu tiên chứa 4 số nguyên N, M, K và Q
- M dòng tiếp theo, mỗi dòng chứa 3 số nguyên U, V và W mô tả có đường nối giữa hai thành phố U, V và khoảng cách giữa hai thành phố là W .
- Dòng tiếp theo chứa K số nguyên cách nhau bởi khoảng trắng là số thứ tự các thành phố diễn ra kì thi.
- Q dòng tiếp theo, mỗi dòng chứa số nguyên X là số thứ tự thành phố mà Timon đang sống.

OUTPUT

- Với mỗi truy vấn X ghi ra file **MCITY.OUT** một số nguyên duy nhất là khoảng cách ngắn nhất tìm được.

RÀNG BUỘC

- $2 \leq N \leq 100000$
- $1 \leq M \leq 100000$
- $1 \leq K \leq N$
- $1 \leq Q \leq 100000$; $1 \leq U \leq N$; $1 \leq V \leq N$; $1 \leq W \leq 10$; $1 \leq X \leq N$

VÍ DỤ

MCITY.INP	MCITY.OUT	Hình minh họa
<pre> 4 4 2 2 1 2 1 2 3 3 3 4 2 4 1 1 2 4 3 2 </pre>	<pre> 2 0 </pre>	

Code tham khảo

```

#include <bits/stdc++.h>
using namespace std;
#define INF LLONG_MAX
#define f first
#define s second
typedef long long ll;
typedef pair<int, int> pii;
vector<vector<pii> > graph;
vector<ll> dist;

void djikstra(int n) {
    priority_queue<pii, vector<pii>, greater<pii> > pq;
    dist.resize(n+1, INF);
    dist[0] = 0;
    pq.push(make_pair(0, 0));
    int u, v, w;
    while(!pq.empty()) {
        u = pq.top().s;
        pq.pop();
        for(int i=0; i<graph[u].size(); i++) {
            v = graph[u][i].f;
            w = graph[u][i].s;
            if(dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push(make_pair(dist[v], v));
            }
        }
    }
}

int main() {
    freopen("MCITY.INP", "r", stdin);
    freopen("MCITY.OUT", "w", stdout);
    int n, m, k, q, u, v, w, x;
    cin >> n >> m >> k >> q;
    graph.resize(n+1);

```

```

for(int i=0;i<m;i++) {
    cin >> u >> v >> w;
    graph[u].push_back(make_pair(v, w));
    graph[v].push_back(make_pair(u, w));
}
while(k--) {
    cin >> x;
    graph[x].push_back(make_pair(0, 0));
    graph[0].push_back(make_pair(x, 0));
}

dijkstra(n);

while(q--) {
    cin >> x;
    if(dist[x] == INF)
        cout << "-1" << endl;
    else
        cout << dist[x] << endl;
}
return 0;
}

```

7.5.2.2. MHIGH – CHI PHÍ TỐI THIỂU ĐỂ LÀM BẰNG ĐƯỜNG ĐI

Bạn được cho một bản đồ thành phố được hệ thống hóa thành ma trận có **N** hàng và **M** cột. Mỗi ô mô tả một nút giao thông, tại nút **(i,j)** sẽ có chiều cao **H[i,j]**. Nhiệm vụ của bạn là tìm chi phí nhỏ nhất để làm con đường từ **(1,1)** đến **(N,M)** được bằng phẳng. Biết rằng từ ô **(i,j)** bạn có thể di chuyển sang 4 ô lân cận và không được di chuyển ra ngoài lưới. Bên cạnh đó chi phí để thay đổi chiều cao từ **x** sang **y** sẽ là **|x-y|**.

INPUT

Dữ liệu vào từ file **MHIGH.INP**

- Dòng đầu tiên chứa hai số nguyên **N,M**
- Tiếp theo là ma trận có **N** hàng và **M** cột, tại hàng **i**, cột **j** là giá trị **H[i,j]** là giá trị chiều cao của nút **(i,j)**

OUTPUT

Ghi ra file **MHIGH.OUT** một số nguyên duy nhất là chi phí nhỏ nhất để làm cho đường đi từ ô **(1,1)** đến ô **(N,M)** được bằng phẳng.

RÀNG BUỘC

- $1 \leq N, M \leq 100$
- $1 \leq H[i,j] \leq 100$

VÍ DỤ

MHIGH.INP	MHIGH.OUT
5 5 1 1 1 1 1 9 9 9 9 1 1 3 3 3 1 1 9 9 9 9 1 1 1 1 1	6

Giải thích

1	1	1	1	1
9	9	9	9	1
1	3	3	3	1
1	9	9	9	9
1	1	1	1	1

Bạn có thể thay đổi độ cao 3 → 1 → tốn chỉ phí 6 để biến đoạn đường trở nên bằng phẳng.

Code tham khảo

```
#include<bits/stdc++.h>
using namespace std;

#define ll long long
#define sp ios_base::sync_with_stdio(false),cin.tie(NULL),cout.tie(NULL)
#define f first
#define s second
#define inf 0x3f3f3f3f
#define pi pair<int,int>

int dir[][2]={{1,0},{0,1},{-1,0},{0,-1}};
int main(){
    freopen("MHIGH.inp", "r", stdin);
    freopen("MHIGH.out", "w", stdout);
    sp;
    int n,m;
    cin >> n >> m;
    int a[n][m];
    for(int i=0; i<n; ++i)
        for(int j=0; j<m; ++j) cin >> a[i][j];
    int ans=INT_MAX,id;
    int b[n][m],c[n][m];
    memset(c,0,sizeof c);
    for(int k=1; k<=100; ++k){
        for(int i=0; i<n; ++i)
            for(int j=0; j<m; ++j){
                b[i][j]=inf;
                c[i][j]=0;
                c[i][j]=abs(k-a[i][j]);
            }
        queue<pi> q;
        b[0][0]=c[0][0];
        q.push({0,0});
        int x,y,dx,dy;
        while(!q.empty()){
            x=q.front().f;
            y=q.front().s;
            q.pop();
            for(int p=0; p<4; ++p){
                dx=x+dir[p][0];
                dy=y+dir[p][1];
                if(dx>=0 && dx<n && dy>=0 && dy<m
                    && b[dx][dy]>b[x][y]+c[dx][dy]){
                    b[dx][dy]=b[x][y]+c[dx][dy];
                    q.push({dx,dy});
                }
            }
        }
    }
}
```

```

    }
    ans=min(ans,b[n-1][m-1]);
}
cout<<ans;
return 0;
}

```

7.5.2.3. MALPHA - Ma trận chữ cái

Cho ma trận có N hàng và M cột, trên mỗi ô tại hàng i , cột j chỉ gồm 1 chữ cái tiếng anh hoa hoặc thường. Nhiệm vụ của bạn là tính khoảng cách ngắn nhất giữa hai ô có tọa độ đã cho là x nguồn, y nguồn và x đích, y đích. Biết rằng:

- Hai ô được xem là kề nhau nếu nó có chung cạnh
- Khoảng cách giữa hai ô được tính bằng trị tuyệt đối của hiệu hai mã ASCII của chúng.

INPUT

Dữ liệu vào từ file **MALPHA.INP**

- Dòng đầu tiên chứa hai số nguyên N, M
- N dòng tiếp theo, mỗi dòng chứa M kí tự chữ cái thường hoặc hoa trong bảng chữ cái tiếng anh
- Dòng thứ $n+1$ chứa hai số nguyên $x_{\text{nguồn}}, y_{\text{nguồn}}$,
- Dòng thứ $n+2$ chứa hai số nguyên $x_{\text{đích}}, y_{\text{đích}}$

OUTPUT

Ghi ra file **MALPHA.OUT** một số nguyên duy nhất là kết quả của bài toán

RÀNG BUỘC

- $1 \leq N, M, x_{\text{nguồn}}, y_{\text{nguồn}}$ và $x_{\text{đích}}, y_{\text{đích}} \leq 10^3$

VÍ DỤ

MALPHA.INP	MALPHA.OUT
2 2 ab cd 2 2 1 1	3

Code tham khảo

```

#include <bits/stdc++.h>
#include <stdio.h>
using namespace std ;
typedef pair<int,int> Pair ;
typedef pair<Pair,int> ppi ;
typedef vector<ppi> vppi ;
#define fi first
#define se second

const int N = 1e3 + 2 ;
const int INF = 1e9 + 7 ;

/******/
struct orden {
    bool operator() ( ppi p1 , ppi p2 ) {
        return p1.second > p2.second ;
    }
}

```

```

    }
} ;

int n , m ;
int c[ N ][ N ] ;
int dx[ 4 ] = { 1 , 0 , -1 , 0 } ;
int dy[ 4 ] = { 0 , 1 , 0 , -1 } ;
int dist[ N ][ N ] ;

inline bool in( int x , int y ) {
    bool a = ( 1 <= x && x <= n ) ;
    bool b = ( 1 <= y && y <= n ) ;
    return a && b ;
}

void dijkstra( Pair node ) {
    for( int i = 0 ; i <= n ; i ++ ) {
        for( int j = 0 ; j <= m ; j ++ ) {
            dist[ i ][ j ] = INF ;
        }
    }
    dist[ node.fi ][ node.se ] = 0 ;
    priority_queue<ppi,vppi,orden> q ;
    q.push( { node , 0 } ) ;
    while( !q.empty() ) {
        ppi t = q.top() ;
        q.pop() ;
        Pair a = t.fi ;
        int da = t.se ;
        if( dist[ a.fi ][ a.se ] < da ) continue ;
        for( int it = 0 ; it < 4 ; it ++ ) {
            Pair b = { a.fi + dx[ it ] , a.se + dy[ it ] } ;
            if (!in(b.fi,b.se)) continue ;
            int cost = abs( c[a.fi][a.se] - c[b.fi][b.se] ) ;
            if (dist[b.fi][b.se] > dist[a.fi][a.se] + cost) {
                dist[b.fi][b.se] = dist[a.fi][a.se] + cost ;
                q.push({b , dist[b.fi][b.se]});
            }
        }
    }
}

/*****/

int main() {
    freopen("MALPHA.INP", "r", stdin);
    freopen("MALPHA.OUT", "w", stdout);
    cin >> n >> m ;
    for( int i = 1 ; i <= n ; i ++ ) {
        for( int j = 1 ; j <= m ; j ++ ) {
            scanf("%c" , &c[i][j]) ;
        }
    }
    int xo , yo , xf , yf ;
    cin >> xo >> yo ;
    cin >> xf >> yf ;
    dijkstra({xo,yo}) ;
    cout << dist[xf][yf] ;

    return 0 ;
}

```

7.6. Cây khung và cây khung nhỏ nhất

7.6.1. Một số khái niệm cơ bản

- Cho G là một đơn đồ thị. Một cây được gọi là **cây khung** của G nếu nó là một đồ thị con của G , chứa tất cả các đỉnh của G , liên thông và không có chu trình.
- **Cây khung nhỏ nhất** trong một đồ thị liên thông có trọng số là một cây khung có tổng trọng số trên các cạnh của nó là nhỏ nhất.

7.6.2. Bài tập cơ bản

7.6.2.1. Phát biểu bài toán - MST – MINIMUM SPANNING TREE

Cho đồ thị vô hướng có trọng số gồm N đỉnh và M cạnh, nhiệm vụ của bạn là tính tổng trọng số trên các cạnh của cây khung nhỏ nhất.

INPUT

Dữ liệu vào từ file **MST.INP**

Dòng đầu tiên chứa hai số nguyên N và M , tương ứng là số đỉnh và số cạnh của đồ thị.

M dòng tiếp theo, mỗi dòng chứa 3 số nguyên a_i , b_i , w_i , mô tả tồn tại đường đi từ a_i đến b_i và trọng số trên cạnh là w_i .

OUTPUT

Ghi ra file **MST.OUT** một số nguyên duy nhất là kết quả của bài toán

RÀNG BUỘC

- $2 \leq N \leq 10000$.
- $1 \leq M \leq 100000$.
- $1 \leq a_i, b_i \leq N$.
- $1 \leq w_i \leq 1000$.

VÍ DỤ

MST.INP	MST.OUT
4 5 1 2 7 1 4 6 4 2 9 4 3 8 2 3 6	19

7.6.2.2. Thuật toán Kruskal

7.6.2.2.1. Giới thiệu

- Thuật toán Kruskal là một thuật toán trong lý thuyết đồ thị để tìm cây bao trùm nhỏ nhất của một đồ thị liên thông có trọng số. Nói cách khác, nó tìm một tập hợp các cạnh tạo thành một cây chứa tất cả các đỉnh của đồ thị và có tổng trọng số các cạnh là nhỏ nhất. Thuật toán Kruskal là một ví dụ của thuật toán tham lam.
- Thuật toán này xuất bản lần đầu tiên năm 1956, bởi Joseph Kruskal.

7.6.2.2.2. Tư tưởng thuật toán

Thuật toán Kruskal dựa trên mô hình xây dựng cây khung nhỏ nhất bằng thuật toán hợp nhất.

- Thuật toán không xét các cạnh với thứ tự tùy ý.
- Thuật toán xét các cạnh theo thứ tự đã sắp xếp theo trọng số.

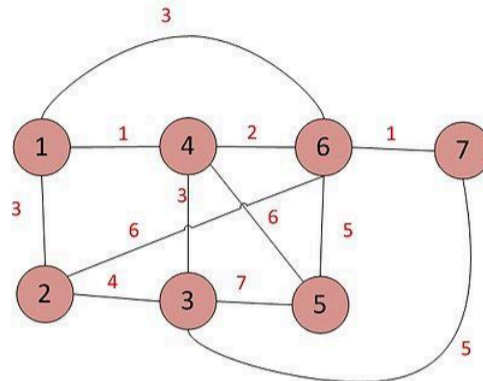
Để xây dựng tập $n-1$ cạnh của cây khung nhỏ nhất - tạm gọi là tập K , Kruskal đề nghị cách kết nạp lần lượt các cạnh vào tập đó theo nguyên tắc như sau:

- Ưu tiên các cạnh có trọng số nhỏ hơn.
- Kết nạp cạnh khi nó không tạo chu trình với tập cạnh đã kết nạp trước đó.

Đó là một nguyên tắc chính xác và đúng đắn, đảm bảo tập K nếu thu đủ $n - 1$ cạnh sẽ là cây khung nhỏ nhất.

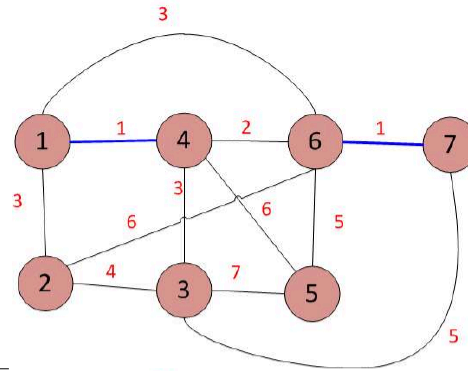
7.6.2.2.3. Mô phỏng thuật toán

Tìm cây khung nhỏ nhất theo thuật toán kruskal của đồ thị

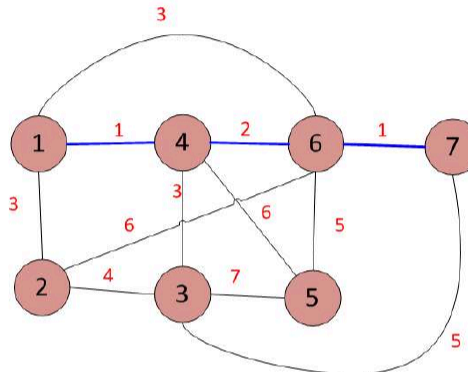


Thứ tự bước	Hình mô tả
Bước 1: 1-4-1: Ta nhận thấy cạnh 1-4 không tạo ra một chu trình nào. Vì vậy, thêm 1-4 vào tập hợp.	

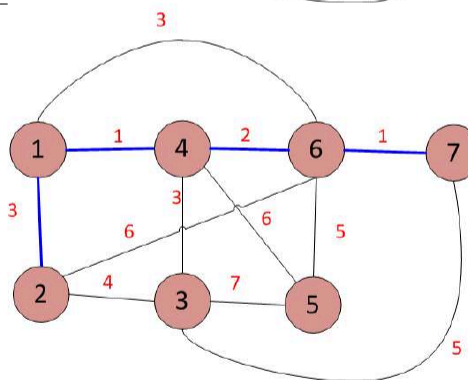
Bước 2: 6-7-1: Ta nhận thấy cạnh 6-7 không tạo ra một chu trình nào. Vì vậy, thêm 6-7 vào tập hợp.



Bước 3: 4-6-2: Ta nhận thấy cạnh 4-6 không tạo ra một chu trình nào. Vì vậy, thêm 4-6 vào tập hợp.

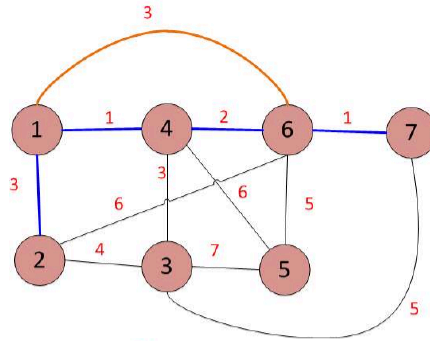


Bước 4: 1-2-3: Ta nhận thấy cạnh 1-2 không tạo ra một chu trình nào. Vì vậy, thêm 1-2 vào tập hợp.

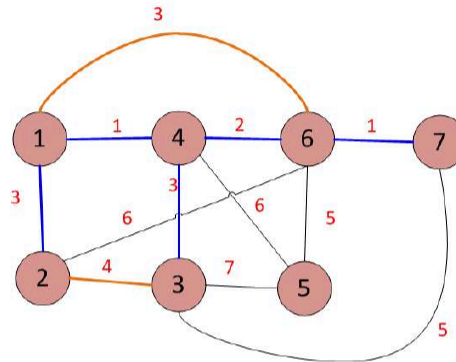


Bước 5:

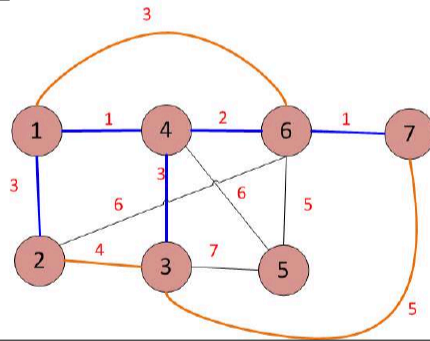
1-6-3: Ta nhận thấy cạnh 1-6 tạo ra một chu trình. Không thêm vào tập hợp.
3-4-3: Ta nhận thấy cạnh 3-4 không tạo ra một chu trình. Vì vậy, thêm 3-4 vào tập hợp.

**Bước 6:**

2-3-4: Ta nhận thấy cạnh 2-3 tạo ra một chu trình. Không thêm vào tập hợp.

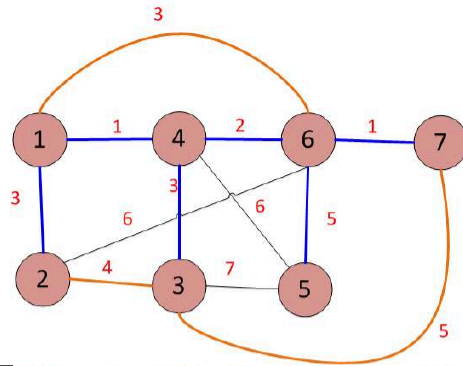
**Bước 7:**

3-7-5: Ta nhận thấy cạnh 3-7 tạo ra một chu trình. Không thêm vào tập hợp.

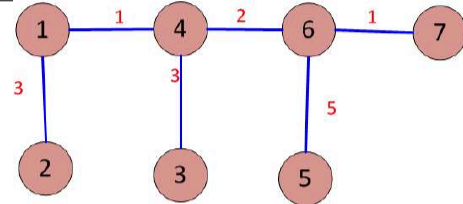


Bước 8:

5-6-5: Ta nhận thấy cạnh 5-6 không tạo ra một chu trình nào. Vì vậy, thêm 5-6 vào tập hợp.

**Đồ thị thu được**

Với tổng chi phí là: Ta cộng tất cả các trọng số giữa các đỉnh lại với nhau
 Vậy tổng chi phí: $3 + 1 + 3 + 2 + 5 + 1 = 15$

**7.6.2.2.4. Code tham khảo**

```
#include <bits/stdc++.h>
#define N 10001
using namespace std;

//par[i]: lưu trữ cha của nút thu i, khởi tạo là chính nó
int par[N];
//Tìm tổ tiên
int anc(int p){
    if (par[p]==p)
        return p;
    else
        return par[p]=anc(par[p]);
}
//Danh dấu 2 đỉnh p,q có cùng tổ tiên --> thuộc cây khung
void join(int p, int q){ par[anc(p)]=anc(q); }

typedef pair<int, int> ii;
typedef pair<int, ii> iii;
#define fr first
#define sc second
vector<iii> edge;
int n, m;

int main(){
    freopen("MST.INP", "r", stdin);
    freopen("MST.OUT", "w", stdout);

    int i, p, q, w, r=0;
    cin >> n >> m;
    for (i=1; i<=n; i++) par[i]=i;
```

```

while (m--){
    cin >> p >> q >> w;
    //Luu tru theo danh sach canh
    edge.push_back(III(w, II(p, q)));
}
//Sap xep lai danh sach canh
sort(edge.begin(), edge.end());
vector<III>::iterator it;
//Duyet tung canh
for (it=edge.begin(); it!=edge.end(); it++){
    //Neu canh dang xet chua duoc bo sung vao cay khung
    if (anc(it->sc.fr) != anc(it->sc.sc)){
        //Bo sung vao cay khung: danh dau 2 dau cua canh thuoc cung to tien
        join(it->sc.fr, it->sc.sc);
        r += it->fr;
    }
}
cout << r << '\n';
return 0;
}

```

7.6.2.2.5. Đánh giá độ phức tạp

- Độ phức tạp thuật toán được cài đặt trên là: $O(\max(E \log E, E \log V))$

7.6.2.3. Thuật toán Prim

7.6.2.3.1. Giới thiệu

- Trong khoa học máy tính, thuật toán Prim là một thuật toán tham lam để tìm cây bao trùm nhỏ nhất của một đồ thị vô hướng có trọng số liên thông. Nghĩa là nó tìm một tập hợp các cạnh của đồ thị tạo thành một cây chứa tất cả các đỉnh, sao cho tổng trọng số các cạnh của cây là nhỏ nhất.
- Thuật toán được tìm ra năm 1930 bởi nhà toán học người Séc Vojtěch Jarník và sau đó bởi nhà nghiên cứu khoa học máy tính Robert C. Prim năm 1957 và một lần nữa độc lập bởi Edsger Dijkstra năm 1959. Do đó nó còn được gọi là thuật toán DJP, thuật toán Jarník, hay thuật toán Prim–Jarník.

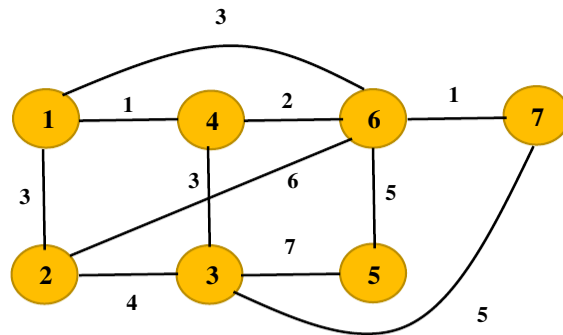
7.6.2.3.2. Tư tưởng thuật toán

Thuật toán xuất phát từ một cây chỉ chứa đúng một đỉnh và mở rộng từng bước một, mỗi bước thêm một cạnh mới vào cây, cho tới khi bao trùm được tất cả các đỉnh của đồ thị.

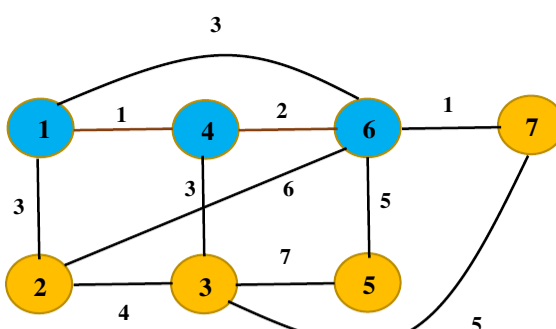
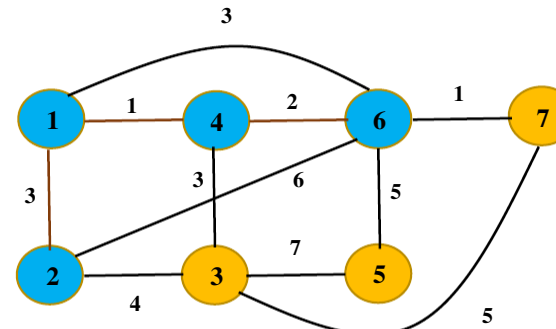
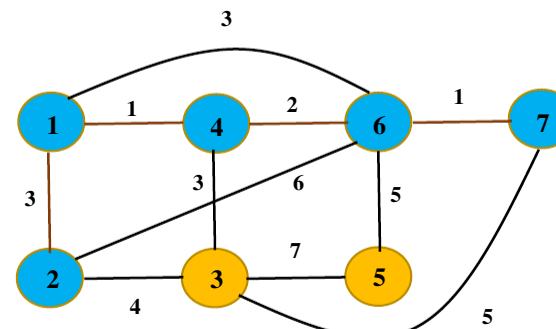
- Dữ liệu vào: Một đồ thị có trọng số liên thông với tập hợp đỉnh V và tập hợp cạnh E (trọng số có thể âm). Đồng thời cũng dùng V và E để ký hiệu số đỉnh và số cạnh của đồ thị.
- Khởi tạo: $V_{\text{mới}} = \{x\}$, trong đó x là một đỉnh bất kì (đỉnh bắt đầu) trong V , $E_{\text{mới}} = \{\}$
- Lặp lại cho tới khi $V_{\text{mới}} = V$:
 - Chọn cạnh (u, v) có trọng số nhỏ nhất thỏa mãn u thuộc $V_{\text{mới}}$ và v không thuộc $V_{\text{mới}}$ (nếu có nhiều cạnh như vậy thì chọn một cạnh bất kì trong chúng)
 - Thêm v vào $V_{\text{mới}}$, và thêm cạnh (u, v) vào $E_{\text{mới}}$
- Dữ liệu ra: $V_{\text{mới}}$ và $E_{\text{mới}}$ là tập hợp đỉnh và tập hợp cạnh của một cây bao trùm nhỏ nhất

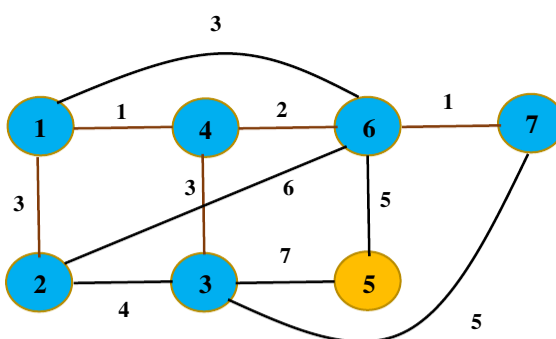
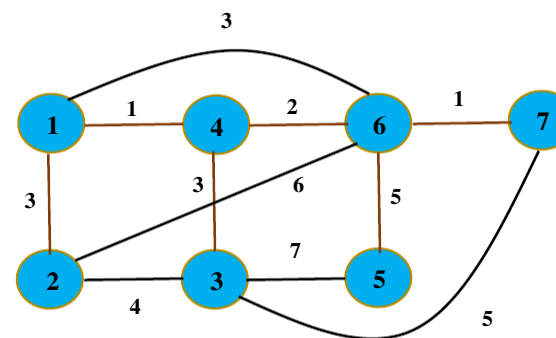
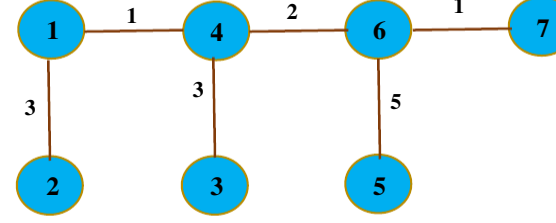
7.6.2.3.3. Mô phỏng thuật toán

Tìm cây khung nhỏ nhất theo thuật toán prim của đồ thị



Thứ tự bước	Hình mô tả
Chọn đỉnh 1 bổ sung vào tập đỉnh mới. Lúc đó các cạnh có nối với đỉnh 1 là $(1,4) : 1$ $(1,2) : 3$ $(1,6) : 3$	
Chọn cạnh có trọng số nhỏ nhất có liên kết với đỉnh 1 là $(1,4)$ bổ sung vào tập cạnh mới. Bổ sung đỉnh 4 vào tập đỉnh mới.	

Các cạnh liên kết với tập đỉnh $\{1,4\}$ gồm: (1,2): 3 (1,6): 3 (4,3): 3 (4,6): 2 Ta chọn cạnh (4,6) bổ sung vào tập cạnh mới, tập đỉnh mới bổ sung đỉnh 6	
Các cạnh liên kết với tập đỉnh $\{1,4,6\}$ gồm: (1,2): 3 (1,6): 3 (4,3): 3 (6,2): 6 (6,5): 5 Ở đây có 3 cạnh trọng số 3, ta có thể chọn 2 cạnh bất kì là (1,2) hoặc (4,3) chứ không thể chọn (1,6) vì cả hai đỉnh đều thuộc tập đỉnh mới. Ta chọn cạnh (1,2) bổ sung vào tập cạnh mới, bổ sung đỉnh 2 vào tập đỉnh mới.	
Tập cạnh có liên kết với tập đỉnh mới $\{1,4,6,2\}$ và đảm bảo có 1 đỉnh thuộc tập đỉnh mới và 1 đỉnh thuộc tập đỉnh của đồ thị chưa được bổ sung vào tập đỉnh của cây khung là (2,3): 4 (4,3): 3 (6,5): 5 (6,7): 1 Ta chọn cạnh (6,7) bổ sung vào tập cạnh mới và bổ sung đỉnh 7 vào tập đỉnh mới.	

Tập cạnh có liên kết với tập đỉnh mới $\{1,4,6,2,7\}$ và đảm bảo có 1 đỉnh thuộc tập đỉnh mới và 1 đỉnh thuộc tập đỉnh của đồ thị chưa được bổ sung vào tập đỉnh của cây khung là (2,3): 4 (4,3): 3 (6,5): 5 (7,3): 5 Ta chọn cạnh (4,3) bổ sung vào tập cạnh mới và bổ sung đỉnh 3 vào tập đỉnh mới.	
Tập cạnh có liên kết với tập đỉnh mới $\{1,4,6,2,7,3\}$ và đảm bảo có 1 đỉnh thuộc tập đỉnh mới và 1 đỉnh thuộc tập đỉnh của đồ thị chưa được bổ sung vào tập đỉnh của cây khung là (6,5): 5 (3,5): 7 Ta chọn cạnh (6,5) bổ sung vào tập cạnh mới và bổ sung đỉnh 5 vào tập đỉnh mới.	
Từ đây ta có đủ 7 đỉnh trong cây khung, suy ra cây khung tìm được.	

7.6.2.3.4. Code tham khảo

```
#include <bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)

using namespace std;
const int MAX = 1e5 + 5;
typedef pair<long long, int> PII;
bool marked[MAX];
//Luu tru danh sach ke
vector <PII> adj[MAX];

long long prim(int u)
{
    priority_queue<PII, vector<PII>, greater<PII> > heap;
```

```

int v;
long long minimumCost = 0;
PII p;
heap.push(make_pair(0, u));
while(!heap.empty())
{
    // Chon canh co trong so nho nhat
    p = heap.top();
    heap.pop();
    u = p.second;
    //Neu dinh u da duoc bo sung vao cay khung thi bo qua
    if(marked[u] == true) continue;
    minimumCost += p.first;
    //Danh dau dinh u duoc chon
    marked[u] = true;
    //Xet tat ca cac dinh lan can voi u
    FOR(i,0,(int) adj[u].size()-1)
    {
        v = adj[u][i].second;
        //Neu v chua duoc xet thi bo sung vao heap
        if(marked[v] == false)
            heap.push(adj[u][i]);
    }
}
return minimumCost;
}

int main()
{
    int n, m, u, v;
    long long c, minimumCost;
    cin >> n >> m;
    FOR(i,0,m-1)
    {
        cin >> u >> v >> c;
        adj[u].push_back(make_pair(c, v));
        adj[v].push_back(make_pair(c, u));
    }
    // Chon dinh bat dau la 1
    minimumCost = prim(1);
    cout << minimumCost << endl;
    return 0;
}

```

7.6.3. Bài tập vận dụng

7.6.3.1. MAXST – CÂY KHUNG CÓ TRỌNG SỐ LỚN NHẤT

Cho đồ thị vô hướng có trọng số gồm N đỉnh và M cạnh, nhiệm vụ của bạn là tính tổng trọng số trên các cạnh của cây khung lớn nhất.

INPUT

Dữ liệu vào từ file **MAXST.INP**

Dòng đầu tiên chứa hai số nguyên N và M , tương ứng là số đỉnh và số cạnh của đồ thị.

M dòng tiếp theo, mỗi dòng chứa 3 số nguyên a_i , b_i , w_i , mô tả tồn tại đường đi từ a_i đến b_i và trọng số trên cạnh là w_i .

OUTPUT

Ghi ra file **MAXST.OUT** một số nguyên duy nhất là kết quả của bài toán

RÀNG BUỘC

- $2 \leq N \leq 10000$.

- $1 \leq M \leq 100000$.
- $1 \leq a_i, b_i \leq N$.
- $1 \leq w_i \leq 1000$.

VÍ DỤ

MAXST.INP	MAXST.OUT
4 5 1 2 7 1 4 6 4 2 9 4 3 8 2 3 6	24

Code tham khảo

```
#include <iostream>
#include <vector>
#include <queue>
#include <functional>
#include <utility>
#include <stdio.h>

using namespace std;
const int MAX = 1e4 + 5;
typedef pair<long long, int> PII;
bool marked[MAX];
vector <PII> adj[MAX];

long long prim(int x)
{
    priority_queue<PII, vector<PII>, greater<PII> > Q;
    int y;
    long long maximumCost = 0;
    PII p;
    Q.push(make_pair(0, x));
    while(!Q.empty())
    {
        // Select the edge with minimum weight
        p = Q.top();
        Q.pop();
        x = p.second;
        // Checking for cycle
        if(marked[x] == true)
            continue;
        maximumCost += p.first;
        marked[x] = true;
        for(int i = 0; i < adj[x].size(); ++i)
        {
            y = adj[x][i].second;
            if(marked[y] == false)
                Q.push(adj[x][i]);
        }
    }
    return maximumCost;
}

int main()
{

```

```

freopen("MAXST.INP","r",stdin);
freopen("MAXST.OUT","w",stdout);
int nodes, edges, x, y;
long long weight, maximumCost;
cin >> nodes >> edges;
for(int i = 0; i < edges; ++i)
{
    cin >> x >> y >> weight;
    adj[x].push_back(make_pair(-weight, y));
    adj[y].push_back(make_pair(-weight, x));
}
// Selecting 1 as the starting node
maximumCost = prim(1);
cout << -maximumCost << endl;
return 0;
}

```

7.6.3.2. SNETWORK – SIMULATE NETWORK

Phòng máy trường Quốc Học có N máy tính được nối với nhau bởi M đoạn cáp. Giữa hai máy có thể có nhiều hơn một đoạn cáp nối, một máy tính có thể nối với chính nó thông qua một đoạn cáp.

Chúng ta nói rằng hai máy tính có thể giao tiếp với nhau nếu nó tồn tại một tập các đoạn cáp kết nối hai máy trực tiếp hoặc gián tiếp.

Mỗi đoạn cáp sẽ có tương ứng một độ trễ. Trường quyết định nâng cấp một số tuyến cáp. Trong kho cáp có sẵn Q đoạn cáp, với mỗi đoạn có độ trễ tương ứng. Nhiệm vụ của bạn là tính xem cần thay thế những đoạn cáp nào (có thể không cần thay thế) từ Q cáp trong kho sao cây khung trên tập N máy của phòng sẽ có độ trễ nhỏ nhất.

INPUT

Dữ liệu vào từ file **SNETWORK.INP**

- Dòng đầu tiên chứa hai số nguyên N và M
- M dòng tiếp theo, mỗi dòng chứa 3 số nguyên U_i , V_i và L_i tương ứng là đoạn cáp thứ i nối hai máy U_i , V_i và độ trễ của đoạn cáp là L_i
- Dòng tiếp theo chứa số nguyên Q
- Dòng tiếp theo chứa Q số, số thứ i mô tả cho giá trị C_i là độ trễ tương ứng với cáp thứ i .

OUTPUT

Ghi ra file **SNETWORK.OUT** một số nguyên duy nhất là độ trễ nhỏ nhất của cây khung trong mạng sau khi đã nâng cấp mạng.

RÀNG BUỘC

- $1 \leq N, M \leq 10^5$
- $1 \leq A, B \leq N$
- $1 \leq L \leq 10^6$
- $0 \leq Q \leq 10^5$
- $1 \leq C[i] \leq 10^6$ với $1 \leq i \leq Q$

VÍ DỤ

SNETWORK.INP	SNETWORK.OUT
4 6	5

1 2 1 1 3 5 1 4 5 1 2 3 2 1 4 2 3 6 5 5 8 2 2 3	
--	--

Code tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
int parent[100005],rnk[100005];
vector<int>mst;
int find(int x)
{
    if(parent[x]!=x)
        parent[x]=find(parent[x]);
    return parent[x];
}
void merge(int x,int y)
{
    int xroot=find(x),yroot=find(y);
    if(rnk[xroot]<rnk[yroot])
        parent[xroot]=yroot;
    else if(rnk[xroot]>rnk[yroot])
        parent[yroot]=xroot;
    else
    {
        parent[yroot]=xroot;
        rnk[xroot]++;
    }
}
void kruskal(const vector<pair<int,pair<int,int>>> &vec)
{
    for(int i=0;i<vec.size();i++)
    {
        int x=vec[i].second.first;
        int y=vec[i].second.second;
        int xroot=find(x);
        int yroot=find(y);
        if(xroot!=yroot)
        {
            mst.push_back(vec[i].first);
            merge(x,y);
        }
    }
}
int main()
{
    freopen("SNETWORK.INP","r",stdin);
    freopen("SNETWORK.OUT","w",stdout);
    int n,m;
    cin>>n>>m;
    vector<pair<int,pair<int,int> > >vec;
    for(int i=1;i<=n;i++)
        parent[i]=i;
    for(int i=0;i<m;i++)
```

```

{
    int a,b,w;
    cin>>a>>b>>w;
    vec.push_back(make_pair(w,make_pair(a,b)));
}
int q;
cin>>q;
int weight[100005];
for(int i=0;i<q;i++)
    cin>>weight[i];
sort(weight,weight+q);
sort(vec.begin(),vec.end());
kruskal(vec);
int i=0,j=0,k=0;
ll ans=0;
//for(auto it:mst)cout<<it<<"\n";
ans=0;i=0;
while(i<mst.size() && j<q && k<n-1)
{
    if(mst[i]<=weight[j])
        ans+=mst[i++];
    else
        ans+=weight[j++];
    k++;
}
while(k<n-1 && i<mst.size()) {ans+=mst[i++];k++;}
cout<<ans<<"\n";
return 0;
}

```

7.6.3.3. TUỔI NƯỚC

Nông dân John quyết định mang nước tới cho N ($1 \leq N \leq 300$) đồng cỏ của mình, để thuận tiện ta đánh số các đồng cỏ từ 1 đến N . Để tưới nước cho 1 đồng cỏ John có thể chọn 2 cách, 1 là đào ở đồng cỏ đó 1 cái giếng hoặc lắp ống nối dẫn nước từ những đồng cỏ trước đó đã có nước tới.

Đào một cái giếng ở đồng cỏ i cần 1 số tiền là W_i ($1 \leq W_i \leq 100,000$). Lắp ống dẫn nước nối 2 đồng cỏ i và j cần 1 số tiền là P_{ij} ($1 \leq P_{ij} \leq 100,000$; $P_{ij} = P_{ji}$; $P_{ii}=0$).

Tính xem nông dân John phải chi ít nhất bao nhiêu tiền để tất cả các đồng cỏ đều có nước.

INPUT

- Dòng 1: Một số nguyên duy nhất: N
- Các dòng 2.. $N+1$: Dòng $i+1$ chứa 1 số nguyên duy nhất: W_i
- Các dòng $N+2$.. $2N+1$: Dòng $N+1+i$ chứa N số nguyên cách nhau bởi dấu cách; số thứ j là P_{ij}

OUTPUT

- Dòng 1: Một số nguyên duy nhất là chi phí tối thiểu để cung cấp nước cho tất cả các đồng cỏ.

VÍ DỤ

TUOINUOC.INP	TUOINUOC.OUT
4	9
5	
4	

4	
3	
0 2 2 2	
2 0 3 3	
2 3 0 4	
2 3 4 0	

GIẢI THÍCH

Có 4 đồng cỏ. Mất 5 tiền để đào 1 cái giếng ở đồng cỏ 1, 4 tiền để đào ở đồng cỏ 2, 3 và 3 tiền để đào ở đồng cỏ 4. Các ống dẫn nước tốn 2, 3, và 4 tiền tùy thuộc vào nó nối đồng cỏ nào với nhau.

Nông dân John có thể đào 1 cái giếng ở đồng cỏ thứ 4 và lắp ống dẫn nối đồng cỏ 1 với tất cả 3 đồng cỏ còn lại, chi phí tổng cộng là $3 + 2 + 2 + 2 = 9$.

Hướng dẫn:

- Xây dựng thêm 1 đỉnh $N+1$ có cạnh nối với i ($i=1..n$) chính là chi phí của giếng nước thứ i .
- Sau đó tìm cây khung có trọng số nhỏ nhất. ở đây có thể dùng Prim hoặc Kruskal đều được.

Code tham khảo

```
//Code by Nguyen Dang Tien Thanh (Chuyen Tin 17-19)
#include <bits/stdc++.h>

using namespace std;

const int MAXN = 305;
const int oo = (int) 1e9 + 7;

int n;
int a[MAXN][MAXN];
int W[MAXN], d[MAXN];
bool avail[MAXN];

void Read() {
    freopen("TUOINUOC.INP", "r", stdin);
    freopen("TUOINUOC.OUT", "w", stdout);
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
        scanf("%d", &W[i]);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++){
            scanf("%d", &a[i][j]);
            if (i == j) a[i][j] = oo;
        }
}

void Init() {
    for (int i = 1; i <= n; i++)
        a[n + 1][i] = a[i][n + 1] = W[i];
    a[n + 1][n + 1] = oo;
    for (int i = 0; i <= n + 1; i++)
        d[i] = oo;
    d[1] = 0;
}
```

```

    memset(avail, true, sizeof(avail));
}

void Prim() {
    do {
        int u = 0;
        for (int i = 1; i <= n + 1; i++)
            if (avail[i] && d[u] > d[i])
                u = i;
        if (u == 0) break;
        avail[u] = false;
        for (int v = 1; v <= n + 1; v++)
            if (avail[v] && d[v] > a[u][v])
                d[v] = a[u][v];
    } while (1);
    int ans = 0;
    for (int i = 1; i <= n + 1; i++)
        ans += d[i];
    printf("%d", ans);
}

int main() {
    Read();
    Init();
    Prim();

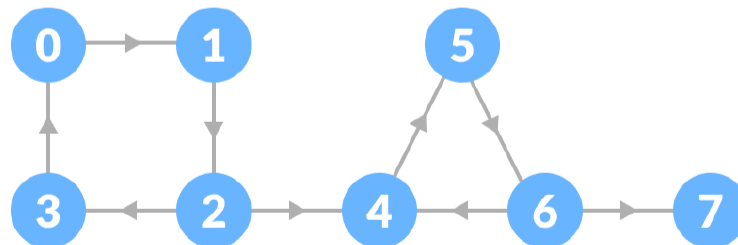
    return 0;
}

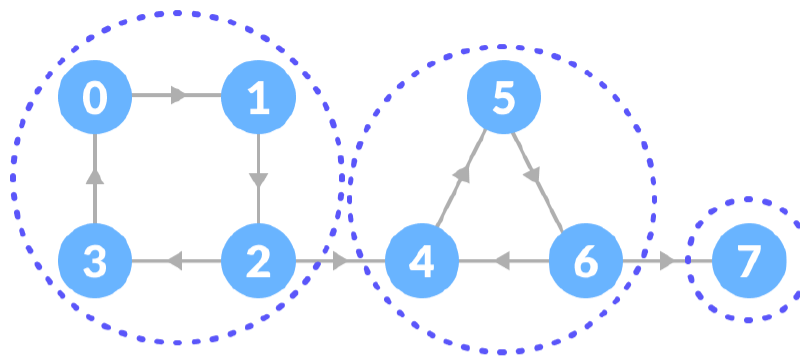
```

7.7. Thành phần liên thông

7.7.1. Khái niệm

Trước hết, khái niệm “thành phần liên thông mạnh” (Strongly Connected Components – SCC) được định nghĩa là một tập hợp các đỉnh trên đồ thị có hướng mà từ 1 đỉnh, ta có thể đi đến tất cả các đỉnh còn lại trong tập hợp đó. Ngoài ra, mỗi TPLTM phải là cực đại, nghĩa là không thể thêm bất cứ đỉnh nào vào một TPLTM mà vẫn tạo thành TPLTM.





7.7.2. Phát biểu bài toán – tarjan - Tìm TPLT mạnh

Cho đồ thị $G(V,E)$ có hướng n đỉnh m cung, Hãy đếm số thành phần liên thông mạnh của G .

Input: Dữ liệu vào từ file **tarjan.inp**.

- Dòng đầu tiên là n,m .
- M dòng tiếp theo mô tả một cung của G .

Output: ghi dữ liệu ra file **tarjan.out**

- Gồm một dòng duy nhất là số TPLT mạnh.

Ràng buộc

- $1 \leq n \leq 100000$
- $1 \leq m \leq 200000$

Ví dụ

tarjan.inp	tarjan.out
3 2 1 2 2 3	3

7.7.3. Thuật toán tarjan

Ý tưởng cơ bản của thuật toán này là: tìm kiếm theo chiều sâu bắt đầu từ một đỉnh tùy ý (và sau đó tìm kiếm sâu dần trên bất kỳ các đỉnh nào chưa được xét). Việc tìm kiếm sẽ không xét đến bất kỳ đỉnh nào đã được xét trước đó. Các thành phần liên thông mạnh tạo nên các cây con của cây tìm kiếm, gốc của những cây con đó chính là gốc của các thành phần liên thông mạnh.

Điểm mấu chốt của thuật toán chính là việc xác định một đỉnh có phải là gốc của một thành phần liên thông mạnh hay không. Đỉnh gốc đơn giản là đỉnh đầu tiên của thành phần liên thông mạnh được tìm thấy trong suốt quá trình tìm kiếm theo chiều sâu. Khi một đỉnh đã được xác định là đỉnh gốc, sau khi thăm xong đỉnh đó, tất cả các đỉnh nằm trong ngăn xếp từ gốc trở lên tạo thành một thành phần liên thông mạnh hoàn chỉnh.

Mô tả cách tìm

int num[] : *num[u]* là thời gian thăm nút *u* trong quá trình duyệt DFS
int Low[]: Khi xuất phát từ *u*, đi qua một số cung trên cây DFS, và nhiều nhất một cung

ngược, ta được một danh sách tập hợp các nút đến được. Low[u] là Num[x] nhỏ nhất trong tất cả các x nằm trong tập này.

7.7.4 Code tham khảo thuật toán tarjan

```
#include <bits/stdc++.h>
#define FOR(i,a,b) for(int i=(a);i<=(b);i++)
#define RESET(a, v) memset(a, (v), sizeof(a))

using namespace std;

const int N = 100005;
const int oo = 0x3c3c3c3c;

int n, m, num[N], low[N],
cnt=0, numSCC=0;
vector<int> a[N];
stack<int> st;

void dfs(int u) {
    low[u] = num[u] = cnt++;
    st.push(u);
    for(int v : a[u]) {
        if (num[v] != -1) low[u] = min(low[u], num[v]);
        else {
            dfs(v);
            low[u] = min(low[u], low[v]);
        }
    }

    if (num[u] == low[u]) {
        numSCC++;
        int v;
        do {
            v = st.top(); st.pop();
            num[v]=low[v]=oo;
        } while (u != v);
    }
}

int main() {

    freopen("tarjan.inp", "r", stdin);
    freopen("tarjan.out", "w", stdout);
    cin >> n >> m;
    int u, v;
    FOR(i,1,m){
        cin >> u >> v;
        a[u].push_back(v);
    }

    RESET(num, -1);
    RESET(low, 0);
    FOR(u,1,n) if (num[u] == -1) dfs(u);

    cout << numSCC;

    return 0;
}
```

7.7.5. Đánh giá độ phức tạp

- Độ phức tạp $O(n+m)$

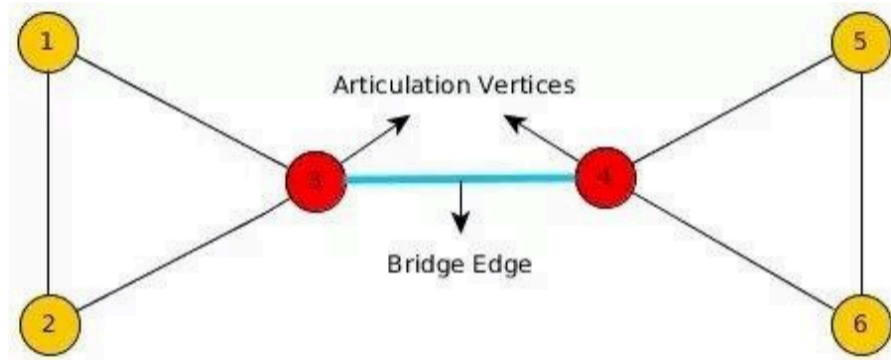
7.8. Khớp và cầu

7.8.1. Khái niệm

(Bạn đọc có thể tham khảo tài liệu tại: <https://codeforces.com/blog/entry/71146>)

Trên đồ thị vô hướng/có hướng, liên thông/liên thông mạnh:

- **Khớp (Articulation Vertices)** là đỉnh mà khi loại nó khỏi đồ thị thì số thành phần liên thông/liên thông mạnh của đồ thị tăng lên.
- **Cầu (Bridge Edge)** là cạnh mà khi loại nó khỏi đồ thị thì số thành phần liên thông/liên thông mạnh của đồ thị tăng lên.



Ví dụ ở hình bên dưới, đỉnh 1 là khớp, lúc bỏ đỉnh 1 đi, đồ thị tách thành hai thành phần liên thông.

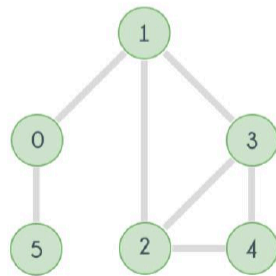


Fig. 1

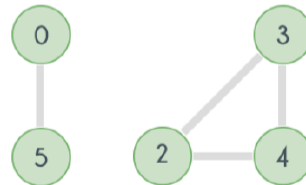


Fig. 2

7.8.2. Bài toán định chiều đồ thị

- Cho đồ thị vô hướng, liên thông, hãy thay mỗi cạnh bằng một cung để được đồ thị có hướng liên thông mạnh.

- Phép định chiều DFS: là thuật toán tìm kiếm theo DFS với bổ sung: mỗi khi xét qua cạnh $(i;j)$ thì thay cạnh $(i;j)$ chỉ bằng một cung $(i;j)$ - nghĩa là phải xóa đi cung $(j; i)$.
- Định lý. Nếu đồ thị vô hướng, liên thông, có mỗi cạnh nằm trên ít nhất một chu trình đơn thì phép định chiều DFS sẽ cho một đồ thị có hướng liên thông mạnh. Một cạnh không nằm trên chu trình đơn nào (cầu) của đồ thị ban đầu sẽ được định hướng thành một cung nối giữa hai thành phần liên thông mạnh.
- Lưu ý. Bài toán định chiều đồ thị được ứng dụng trong giao thông: Định một chiều đi cho mỗi con đường của hệ các con đường hai chiều, vẫn bảo đảm đường đi giữa hai điểm tùy ý.

7.8.3. Bài toán đếm số lượng khớp, cầu

Xét đơn đồ thị vô hướng $G = (V, E)$ có n đỉnh và m cạnh. Người ta định nghĩa một đỉnh gọi là khớp nếu như xóa đỉnh đó sẽ làm tăng số thành phần liên thông của đồ thị. Tương tự như vậy, một cạnh được gọi là cầu nếu xóa cạnh đó sẽ làm tăng số thành phần liên thông của đồ thị.

Vấn đề đặt ra là cần phải đếm tất cả các khớp và cầu của đồ thị G .

Input: dữ liệu vào từ file **khopcau.inp**

- Dòng đầu: chứa hai số tự nhiên n, m .
- M dòng sau mỗi dòng chứa một cặp số (u, v) ($u < v, 1 \leq u \leq n, 1 \leq v \leq n$) mô tả một cạnh của G .

Output: ghi ra file **khopcau.out**

Gồm một dòng duy nhất ghi hai số, số thứ nhất là số khớp, số thứ hai là số cầu của G

Ràng buộc

- $1 \leq n \leq 10000$
- $1 \leq m \leq 50000$

Ví dụ

khopcau.inp	khopcau.out
10 12 1 10 10 2 10 3 2 4 4 5 5 2 3 6 6 7 7 3 7 8 8 9 9 7	4 3

7.8.4. Ý tưởng

Cách giải: Tương tự thuật toán Tarjan (tìm thành phần liên thông mạnh)

Ý nghĩa các mảng $num[N]$ và $low[N]$ như sau:

- **num[u]** là số thứ tự duyệt đến đỉnh u ;

- **low[u]** là *giá trị nhỏ nhất* của các giá trị **num[j]** với j là đỉnh tới được từ một đỉnh i nào đó của nhánh DFS gốc u bằng một cung ngược.

Vậy Low[u] phản ánh cung ngược về phía gốc lên *cao nhất* xuất phát từ những đỉnh thuộc nhánh DFS gốc u . Nếu không có cung ngược thì Low[u] = ∞ .

Dựa vào việc định chiều đồ thị, có thể phát hiện các khớp và cầu như sau:

- **(u;v)** là **cầu** nếu và chỉ nếu **Low[v] $\geq$ Number[v]** nghĩa là không có cung ngược từ một đỉnh thuộc nhánh DFS gốc v lên phía trên v .
- **u** là **khớp** nếu và chỉ nếu xảy ra một trong hai trường hợp:
 - **u** không là gốc của cây DFS và **Low[v] $\geq$ Number[u]** nghĩa là các cung ngược từ một đỉnh thuộc nhánh DFS gốc v không vượt quá đỉnh u .
 - **u** là gốc cây DFS, có từ 2 nhánh con trở lên.

7.8.5. Code tham khảo bài toán đếm số lượng khớp cầu

```
#include <stdio.h>
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

const int N = 100005;
int n, m;
vector<int> a[N];
int CriticalEdge = 0;
bool CriticalNode[N];
int Num[N], Low[N], Time = 0;

void visit(int u, int p) {
    int NumChild = 0;
    Low[u] = Num[u] = ++Time;
    for (int v : a[u])
        if (v != p) {
            if (Num[v] != 0)
                Low[u] = min(Low[u], Num[v]);
            else {
                visit(v, u);
                NumChild++;
                Low[u] = min(Low[u], Low[v]);

                if (Low[v] >= Num[v])
                    CriticalEdge++;

                if (u == p) {
                    if (NumChild >= 2)
                        CriticalNode[u] = true;
                } else {
                    if (Low[v] >= Num[u])
                        CriticalNode[u] = true;
                }
            }
        }
}

int main() {
    freopen("khopcau.inp", "r", stdin);
    freopen("khopcau.out", "w", stdout);
```

```

scanf("%d%d", &n, &m);
for (int i = 1; i <= m; i++) {
    int x, y;
    scanf("%d%d", &x, &y);
    a[x].push_back(y);
    a[y].push_back(x);
}
for (int i = 1; i <= n; i++)
    if (!Num[i]) visit(i, i);

int Count = 0;
for (int i = 1; i <= n; i++)
    if (CriticalNode[i]) Count++;
printf("%d %d\n", Count, CriticalEdge);
}

```

7.8.6. Đánh giá độ phức tạp về thời gian:

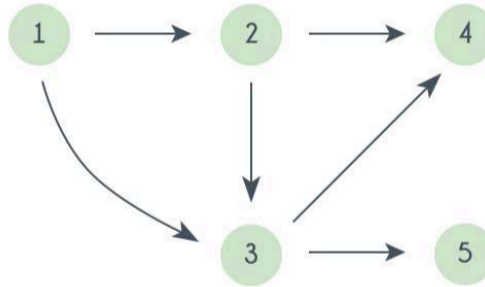
Tương tự như Tarjan, độ phức tạp thuật toán tìm khớp cầu $O(V+E)$

7.9. Sắp xếp tô pô

7.9.1. Khái niệm

- Trong khoa học máy tính, **thứ tự tô pô** của một đồ thị có hướng là một thứ tự sắp xếp của các đỉnh sao cho với mọi cung từ **u** đến **v** trong đồ thị, **u** luôn nằm trước **v**.
- Thuật toán để tìm thứ tự tô pô gọi là thuật toán sắp xếp tô pô. Thứ tự tô pô tồn tại khi và chỉ khi đồ thị có hướng không có chu trình (viết tắt là **DAG** - tiếng Anh **directed acyclic graph**).
- Đồ thị có hướng không có chu trình luôn có ít nhất một thứ tự tô pô, và có thuật toán để tìm thứ tự tô pô trong thời gian tuyến tính.

Ví dụ như đồ thị được cho như bên dưới



Ta có thể dễ dàng thấy thứ tự sắp xếp tô pô : **1, 2, 3, 4, 5**.

7.9.2. Phát biểu bài toán

Cho đồ thị có hướng không có chu trình gồm **n** đỉnh và **m** cạnh, bạn hãy in ra thứ tự sắp xếp tô pô của chúng.

INPUT: Dữ liệu vào từ file **TOPOSORT.INP** gồm

- Dòng đầu tiên chứa hai số nguyên **n, m**
- m** dòng tiếp theo, mỗi dòng chứa hai số nguyên **u, v** mô tả có cạnh nối từ **u** đến **v**

OUTPUT: Ghi ra file **TOPOSORT.OUT** thứ tự sắp xếp tô pô của **n** đỉnh, trong trường hợp có nhiều thứ tự sắp xếp tô pô, bạn hãy in ra thứ tự nhỏ nhất theo từ điển.

RÀNG BƯỚC

- $1 \leq n \leq 10$
- $1 \leq m \leq 20$.
- $1 \leq u, v \leq n$

VÍ DỤ

TOPOSORT.INP	TOPOSORT.OUT
5 6 1 2 1 3 2 3 2 4 3 4 3 5	1 2 3 4 5

7.9.3. Ý tưởng thuật toán

Cách làm thứ nhất:

- Trong đồ thị có hướng không chu trình thì luôn tồn tại ít nhất một đỉnh có bậc vào bằng 0 (có thể chứng minh bằng phản chứng). Lần lượt lấy các đỉnh bậc vào bằng 0 ra khỏi đồ thị, cho xếp ở vị trí đầu tiên, bỏ các cung ra từ những đỉnh bậc 0 này thì đồ thị sẽ xuất hiện những đỉnh bậc 0 mới. lại lấy những đỉnh bậc 0 này xếp vào vị trí tiếp theo. Cứ làm như vậy cho đến khi bỏ hết các đỉnh của đồ thị.

Cách làm thứ hai (đơn giản và dễ cài đặt hơn):

- Thực hiện quá trình duyệt DFS đồ thị, đỉnh nào được duyệt xong sớm hơn sẽ nằm ở vị trí cuối hơn hay có chỉ số lớn hơn

7.9.4. Code tham khảo

7.9.4.1. Vận dụng giải thuật duyệt sâu

```
#include<bits/stdc++.h>
using namespace std;

void dfs(int src,vector<int> v[],bool vis[],stack<int> &s){
    vis[src]=true;
    for(auto it=v[src].begin();it!=v[src].end();++it){
        if(!vis[*it])
            dfs(*it,v,vis,s);
    }
    s.push(src);
}

int main(){
    freopen("toposort.inp","r",stdin);
    freopen("toposort.out","w",stdout);

    int n,m;
    cin>>n>>m;
    vector<int> v[n];
    int x,y;
    // Lưu đồ thị sử dụng danh sách kề
    for(int i=0;i<m;i++){
        cin >> x >> y;
        v[x-1].push_back(y-1);
    }
}
```

```

//sap xep danh sach ke theo thu tu dinh giam dan
int h;
for(int i=0;i<n;i++){
    h = v[i].size();
    if(h!=0) sort(v[i].begin(),v[i].end(),greater<int>());
}
//khởi tạo
bool vis[n]={false};
stack<int> s;
//Duyệt sau xác định thu tu dinh duoc tham
for(int i=n-1;i>=0;i--){
    if(!vis[i])
        dfs(i,v,vis,s);
}
/*
    Sau khi duyệt sau, lấy thu tu dinh tu duoi len
    Qua trình duyệt sau, đã đẩy các dinh duyệt vào stack
*/
while(!s.empty()){
    cout<<s.top()+1<<" ";
    s.pop();
}
cout<<endl;
}

```

7.9.4.2. Vận dụng giải thuật duyệt rộng

```

#include <bits/stdc++.h>
using namespace std;

int main()
{
    freopen("toposort.inp","r",stdin);
    freopen("toposort.out","w",stdout);

    int n,m;
    cin>>n>>m;
    priority_queue<int,vector<int>,greater<int>>> q;
    vector<int> adj[n+1];
    bool vis[n+1] = {false};
    //Luu bac cua do thi
    int indeg[n+1] = {0};
    int x,y;
    //Doc du lieu
    while(m-->0)
    {
        cin >> x >> y;
        adj[x].push_back(y);
        indeg[y]=indeg[y]+1;
    }
    //Duyệt từng dinh trong do thi
    for(int i=1;i<=n;i++)
    {
        // Nhung dinh nao ko co canh vào và chưa tham thi đẩy vào queue
        if(indeg[i]==0 && vis[i]==false) q.push(i);
        //Trong khi queue khác rỗng thì
        while(!q.empty())
        {
            //lay dinh ra và hiện thi dinh
            int t=q.top();
            cout<<t<<" ";
            vis[t]=true;

```

```

q.pop();
//Voi moi lan can cua dinh duoc lay ra thi
for(int i=0;i<adj[t].size();i++)
{
    //giam bac cua cac dinh con (loai bo canh (t,i))
    indeg[adj[t][i]]--;
    /*
        Neu dinh con sau khi loai dinh cha
        tro thanh khong co cung vao thi bo sung dinh vao queue
    */
    if(indeg[adj[t][i]]==0) q.push(adj[t][i]);
}
}
return 0;
}

```

7.9.5. Đánh giá độ phức tạp thuật toán

Độ phức tạp khi lưu trữ theo danh sách kề : $O(V+E)$

7.10. Luồng cực đại

7.10.1. Khái niệm

7.10.1.1. Mạng (flow network)

- **Mạng $G = (V, E)$** là đồ thị có hướng, trong đó mỗi cung $(u,v) \in E$ được gán 1 giá trị không âm, gọi là khả năng thông qua (băng thông) $c(u, v) \geq 0$.
- Nếu $(u,v) \notin E$, thì đặt $c(u,v) = 0$.
- Trong mạng chứa 2 đỉnh phân biệt: đỉnh phát s và đỉnh thu t .

7.10.1.2. Luồng (flow)

Cho mạng $G = (V, E)$ có đỉnh phát: s , đỉnh thu: t

Luồng f trong mạng G là một ánh xạ $f : V \times V \rightarrow \mathbb{R}^+$ thỏa mãn điều kiện:

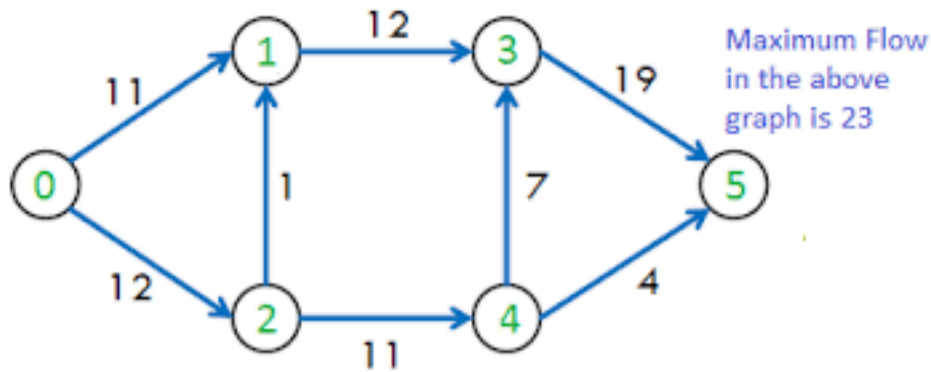
- $\forall u,v \in V: f(u, v) \leq c(u,v)$.
- $\forall v \in V \setminus \{s, t\}$: tổng giá trị luồng vào bằng tổng giá trị luồng ra tại v .

7.10.1.3. Một số khái niệm khác

- Giá trị $f(u, v)$ được gọi là **lưu lượng thực** của luồng (net flow) từ đỉnh u đến đỉnh v
- **Giá trị của luồng** được xác định bởi tổng **lưu lượng thực** từ đỉnh phát đến tất cả các đỉnh trong mạng:

$$|f| = \sum_{v \in V} f(s, v)$$

- **Luồng cực đại (max-flow)** trong mạng $G=(V,E)$ là luồng mà có giá trị luồng là lớn nhất.



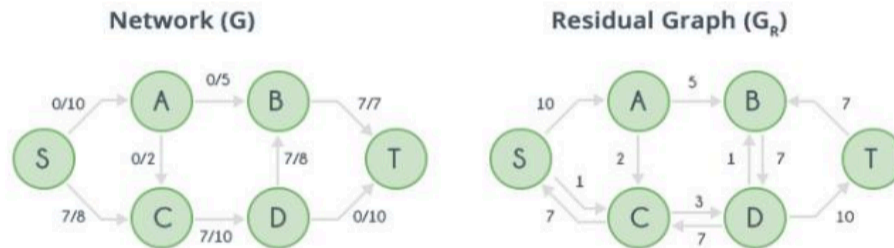
7.10.1.4. Mạng thặng dư (Residual network)

Mạng thặng dư chứa các cung có thể thay đổi để tăng lưu lượng thực trong luồng.

Xét mạng $G = (V, E) \Rightarrow$ mạng thặng dư: $G_r = (V, E_r)$ là đồ thị có hướng, mỗi cung $(u, v) \in E_r$ có trọng số $cr(u, v) \in \mathbb{R}^+$:

- $cr(u, v) = c(u, v) - f(u, v)$ // Lượng băng thông có thể nhận thêm trước khi đạt đến giới hạn.
- $cr(v, u) = f(u, v)$ // Chiều ngược lại = lượng tối đa có thể giảm bớt.
- Nếu $cr(e) = 0$: bỏ qua cung e.

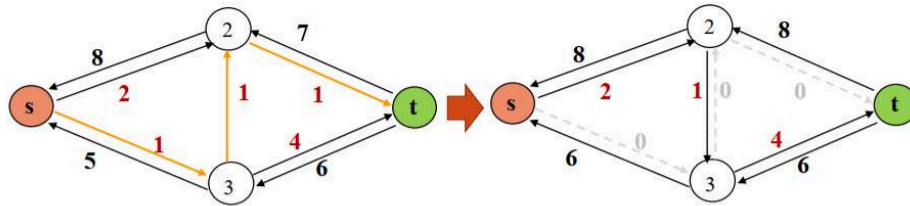
Ví dụ



7.10.1.5. Đường tăng luồng (augmenting path)

- Một đường tăng luồng là một đường đi từ s đến t trong mạng thặng dư.
- Giá trị tăng của mỗi đường tăng luồng là trọng số nhỏ nhất trong các cung thuộc đường đi - $\delta(P)$.
- Để tăng luồng dọc theo đường P: ta sẽ thêm $\delta(P)$ vào các cung trên đường P và thay đổi giá trị mạng thặng dư tương ứng:
 - ✓ $cr(u, v) = cr(u, v) - \delta(P) \quad \forall (u, v) \in P$
 - ✓ $cr(v, u) = cr(v, u) + \delta(P) \quad \forall (u, v) \in P$

Ví dụ cho mạng G như hình bên dưới



7.10.2. Phát biểu bài toán

Cho một mạng luồng, hãy tìm giá trị luồng cực đại.

INPUT: dữ liệu vào từ file **maxflow.inp**

Dòng đầu tiên chứa 4 số nguyên dương **n, m, s, t**, ($2 \leq n \leq 1000$) tương ứng là số đỉnh, số cạnh của đồ thị, chỉ số của đỉnh phát và đỉnh thu.

m dòng tiếp theo, mỗi dòng có dạng ba số **u, v, c** cách nhau ít nhất một dấu cách thể hiện có cung **u, v** trong mạng với khả năng thông qua là **c** ($1 \leq c \leq 10^6$).

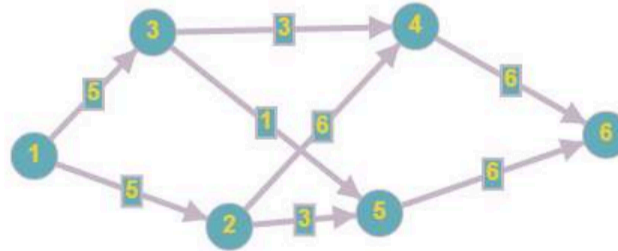
OUTPUT: Ghi ra file **maxflow.out**

In ra một số duy nhất là giá trị của luồng cực đại trên mạng.

VÍ DỤ

maxflow.inp	maxflow.out
6 8 1 6 1 2 5 1 3 5 2 4 6 2 5 3 3 4 3 3 5 1 4 6 6 5 6 6	9

Hình minh họa



7.10.3. Ý tưởng thuật toán Ford-Fulkerson

Bước 1: Khởi tạo:

Một luồng bất kỳ trên mạng, chẳng hạn như luồng 0 (luồng trên các cung đều bằng 0).

Bước 2: Lặp hai bước sau:

- ✓ **Tìm đường tăng luồng P** đối với luồng hiện có: Tìm đường đi cơ bản từ A tới B trên đồ thị tăng luồng, nếu không tìm được đường tăng luồng thì bước lặp kết thúc.
- ✓ Tăng luồng dọc theo đường P

Bước 3: Thông báo giá trị luồng cực đại tìm được.

Có nhiều cách tìm đường tăng luồng. Cách đơn giản nhất là chọn đường đi có ít cạnh nhất, cách này cho ta thuật toán của **Edmond-Karp**. Một cách hiệu quả hơn là phân tầng đồ thị theo khoảng cách tới đỉnh phát và chỉ quan tâm đến những đường đi mà các đỉnh thuộc các tầng khác nhau, đây gọi là thuật toán **Dinic**.

7.10.4. Code tham khảo

7.10.4.1. Thuật toán Edmond-Karp

Độ phức tạp thuật toán : $O(V \cdot E^2)$

```
#include <stdio.h>
#include <queue>
using namespace std;

#define long long long
#define NMAX 2309
#define MMAX 12309
#define oo 1000111000
void minimize(int &a, int b){ if (a>b) a=b; }
/*
    n: số đỉnh
    m: số cạnh
    source: nút nguồn
    target: nút đích
    a: danh sách kề
*/
int n, m, source, target;
vector<int> a[MMAX];
int c[NMAX][NMAX];
int f[NMAX][NMAX];
int d[MMAX];

/*
Function kiểm tra xem có đường tăng luồng không?
Nếu có đường tăng luồng thì sẽ tồn tại đường đi từ start đến target
*/
bool findpath(int start, int target){
    queue<int> qu;
    int i, u, v;
    for (i=1; i<=n; i++) d[i]=0;
    d[start] = oo;
    qu.push(start);

    while (qu.size()){
        u=qu.front(); qu.pop();
        if (u==target) return true;
        for (i=0; v=a[u][i]; i++){
            if (d[v]==0 && c[u][v] > f[u][v]){
                d[v]=u;
                qu.push(v);
            }
        }
    }
    return false;
}

void enlarge(){
    int u, v, delta=oo;
    //Tìm giá trị tăng luồng nhỏ nhất
    for (v=target; (u=d[v])!=oo; v=u)
        minimize(delta, c[u][v] - f[u][v]);
```

```

        //cập nhật giá trị tăng luồng mới
        for (v=target; v!=source; v=u){
            u=d[v];
            f[u][v] += delta;
            f[v][u] -= delta;
        }
    }

    long answer(int u){
        int i, v;
        long r=0;
        for (i=0; v=a[u][i]; i++){
            r += f[u][v];
        }
        return r;
    }

    main(){
        int i, p, q, w;
        //Đọc dữ liệu
        scanf("%d%d%d", &n, &m, &source, &target);
        for (i=1; i<=m; i++){
            scanf("%d%d%d", &p, &q, &w);
            //Đồ thị lưu theo dạng danh sách kề
            a[p].push_back(q);
            a[q].push_back(p);
            //Trọng số sử dụng ma trận để lưu
            c[p][q]=w;
        }
        for (i=1; i<=n; i++) a[i].push_back(0);
        while (findpath(source, target)) enlarge();

        printf("%lld\n", answer(source));
    }

```

7.10.4.1. Thuật toán Dinic:

Độ phức tạp thuật toán: $O(V^2 \cdot E)$

```

#include <bits/stdc++.h>

using namespace std;

const int INF = 1e9;
/*
Định nghĩa cạnh:
u: đỉnh bắt đầu
v: đỉnh kết thúc
capacity: trọng số cạnh
flow: giá trị luồng
*/
struct Edge {
    int u, v;
    int capacity;
    int flow;
};
/*
Định nghĩa một mạng:
n: số nút trong mạng
source: đỉnh nguồn
sink: đỉnh đích
a: mảng 2 chiều: đệm tăng của đỉnh
E: Danh sách cạnh
cur: nhãn đỉnh trên đồ thị ban đầu
dist: nhãn đỉnh trên đồ thị tăng (level graph)
*/

```

```

*/
struct Network {
    int n;
    int source, sink;

    vector<vector<int>> > a;
    vector<Edge> E;

    vector<int> cur;
    vector<int> dist;

    Network() {}
    Network(int n, int s, int t) {
        this->n = n;
        source = s; sink = t;
        a = vector<vector<int>> > (n + 1);
        cur = dist = vector<int> (n + 1);
    }

    void addEdge(int u, int v, int c) {
        a[u].push_back(E.size()); E.push_back({u, v, c, 0});
        a[v].push_back(E.size()); E.push_back({v, u, 0, 0});
    }
    /*
        Xác định có đường tăng luồng hay không?
    */
    bool bfs() {
        fill(dist.begin(), dist.end(), -1);
        queue<int> Q; Q.push(source); dist[source] = 0;
        while (!Q.empty()) {
            int u = Q.front(); Q.pop();
            for (int i = 0; i < a[u].size(); ++i) {
                int id = a[u][i], v = E[id].v;
                if (dist[v] < 0 && E[id].flow < E[id].capacity) {
                    dist[v] = dist[u] + 1;
                    Q.push(v);
                }
            }
        }
        return dist[sink] >= 0;
    }

    int dfs(int u, int flow) {
        if (!flow) return 0;
        if (u == sink) return flow;
        for (; cur[u] < a[u].size(); ++cur[u]) {
            int id = a[u][cur[u]], v = E[id].v;
            if (dist[v] != dist[u] + 1) continue;
            int delta = dfs(v, min(flow, E[id].capacity - E[id].flow));
            if (delta) {
                E[id].flow += delta; E[id ^ 1].flow -= delta;
                return delta;
            }
        }
        return 0;
    }

    int maxFlow() {
        int ans = 0;
        //Trong khi còn đường tăng luồng trong đồ thị
        while (bfs()) {
            fill(cur.begin(), cur.end(), 0);

```

```

        while (true) {
            int delta = dfs(source, INF);
            if (!delta) break;
            ans += delta;
        }
        return ans;
    }
};

int main() {
    freopen("dinic.inp", "r", stdin);
    freopen("dinic.out", "w", stdout);

    ios::sync_with_stdio(false);
    int n, m, s, t;
    //Nhập đồ thị
    cin >> n >> m >> s >> t;
    Network G (n, s, t);
    while (m--) {
        int u, v, c;
        cin >> u >> v >> c;
        G.addEdge(u, v, c);
    }
    //Tìm giá trị luồng cực đại
    cout << G.maxFlow() << endl;
    return 0;
}

```

7.11. Cặp ghép

7.11.1. Phát biểu bài toán

Có x người nam, y người nữ và m mối liên hệ có cảm tình giữa hai người nam và nữ. Bạn hãy xác định số cách ghép cặp tối đa có thể thực hiện, biết rằng mỗi người nam hay nữ chỉ được ghép cặp một lần.

INPUT: Dữ liệu vào từ file **capghep.inp** gồm:

- Dòng đầu tiên chứa hai số x, y, m ($x, y \leq 1000$), theo thứ tự là số bạn nam, số bạn nữ và m mối liên hệ có cảm tình.
- m dòng tiếp theo mỗi dòng ghi hai số i, j cách nhau bởi một dấu cách thể hiện có hai bạn x_i, y_j có cảm tình với nhau.

OUTPUT: ghi ra file **capghep.out** số lượng cặp ghép tối đa có thể ghép.

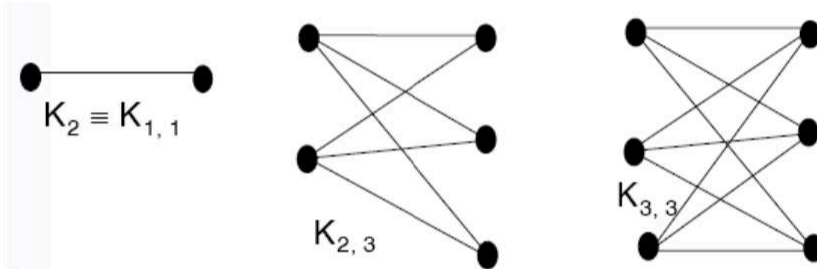
VÍ DỤ

capghep.inp	capghep.out
4 5 9 1 1 1 4 2 1 2 2 2 4 3 2 3 3 4 2	4

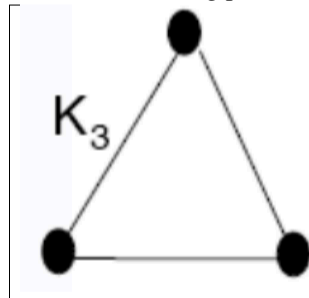
7.11.2. Một số khái niệm

7.11.2.1. Đồ thị hai phía

- Trong Lý thuyết đồ thị, **đồ thị hai phía** (đồ thị lưỡng phân hay đồ thị hai phần) (tiếng Anh: **bipartite graph**) là một đồ thị đặc biệt, trong đó tập các đỉnh có thể được chia thành hai tập không giao nhau thỏa mãn điều kiện không có cạnh nối hai đỉnh bất kỳ thuộc cùng một tập.
- Những tính chất của đồ thị hai phía được đề cập đến đầu tiên bởi Dénes Kőnig(1916). Dénes Kőnig cũng là người viết cuốn sách đầu tiên Theorie der endlichen und unendlichen Graphen 1936 về Lý thuyết đồ thị.
- Đồ thị hai phía xuất hiện trong các bài toán dùng đồ thị biểu diễn quan hệ hai ngôi giữa hai tập A và tập B không giao nhau. Một ví dụ cho quan hệ này là quan hệ hôn nhân giữa hai tập hợp người nam và nữ.
- Ví dụ đồ thị hai phía



- Ví dụ về đồ thị không phải là đồ thị hai phía



K_3 không phải là đồ thị lưỡng phân vì nếu ta chia các đỉnh của nó thành 2 phần rời nhau thì một trong 2 phần này phải chứa 2 đỉnh. Nếu đồ thị là lưỡng phân thì các đỉnh này không thể nối với nhau bằng một cạnh. Nhưng trong K_3 mỗi đỉnh được nối với đỉnh khác bằng một cạnh.

7.11.2.2. Kiểm tra một đồ thị có phải là đồ thị hai phía hay không

7.11.2.2.1. Phát biểu bài toán – cbipartite

Cho đồ thị gồm n đỉnh, m cạnh. Bạn hãy xác định đồ thị đã cho có phải là đồ thị hai phía hay không.

INPUT: Dữ liệu vào từ file **cbipartite.inp**

- Dòng đầu tiên gồm 2 số nguyên n và m

- **m** dòng tiếp theo, mỗi dòng gồm hai số nguyên **u, v** mô tả có cạnh nối giữa hai đỉnh **u** và **v**.

OUTPUT: Ghi ra file **cbipartite.out**

Nếu đồ thị đã cho là đồ thị hai phía in ra phía của đỉnh theo thứ tự từ **1** đến **n**, trường hợp đồ thị không phải là đồ thị hai phía, in ra “**Invalid**”.

RÀNG BUỘC

- $1 \leq n, m \leq 20$

VÍ DỤ

cbipartite.inp	cbipartite.out
3 3 1 2 2 3 1 3	Invalid
5 6 1 2 1 4 1 5 3 2 3 4 3 5	1 2 1 2 2

7.11.2.2.2. Code tham khảo

```
#include <stdio.h>
#include <vector>
#include <iostream>
#include <algorithm>
using namespace std;

int n, m;
vector<int> a[12309];
bool Invalid = false;
bool Visited[12309];
int Color[12309];

void visit(int u) {
    Visited[u] = true;
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i];
        if (!Visited[v]) {
            Color[v] = 3 - Color[u];
            visit(v);
        }
        else {
            if (Color[v] == Color[u])
                Invalid = true;
        }
    }
}

int main() {
    freopen("cbipartite.inp", "r", stdin);
    freopen("cbipartite.out", "w", stdout);
```

```

cin >> n >> m;
for (int i = 1; i <= m; i++) {
    int p, q;
    cin >> p >> q;
    a[p].push_back(q);
    a[q].push_back(p);
}

for (int i = 1; i <= n; i++) {
    if (!Visited[i]) {
        Color[i] = 1;
        visit(i);
    }
}

if (Invalid)
    cout << "Invalid" << endl;
else {
    for (int i = 1; i <= n; i++) cout << Color[i] << " ";
    cout << endl;
}
}

```

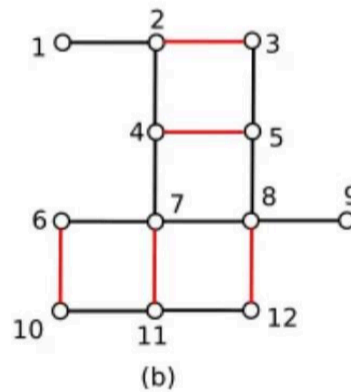
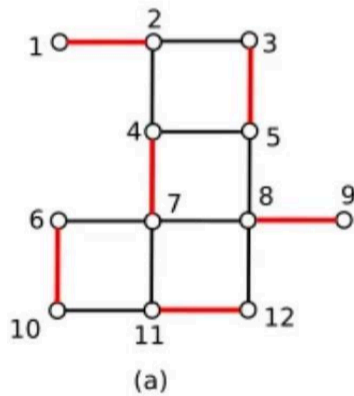
7.11.2.2.2. Đánh giá độ phức tạp

Độ phức tạp giải thuật $O(V+E)$

7.11.2.3. Cặp ghép

Một cặp ghép (matching) của một đồ thị $G(V, E)$ là một tập cạnh M sao cho không có hai cạnh nào của M có chung một đầu nút.

Ví dụ



7.11.2.4. Cặp ghép tối đại (Maximal matching)

Cặp ghép tối đại là cặp ghép mà ta không thể thêm cạnh nào của đồ thị vào M sao cho tập cạnh sau khi thêm vẫn là một cặp ghép.

Ví dụ cả hai cặp ghép (a) và (b) ở ví dụ trên đều là cặp ghép tối đại.

7.11.2.5. Cặp ghép cực đại (Maximum matching)

Cặp ghép cực đại là cặp ghép có số cạnh nhiều nhất. Cặp ghép cực đại cũng là cặp ghép tối đại nhưng điều ngược lại chưa chắc đã đúng.

Ở ví dụ trên chỉ có ví dụ (a) là cặp ghép cực đại.

7.11.2.6. Đỉnh đã ghép, đỉnh chưa ghép

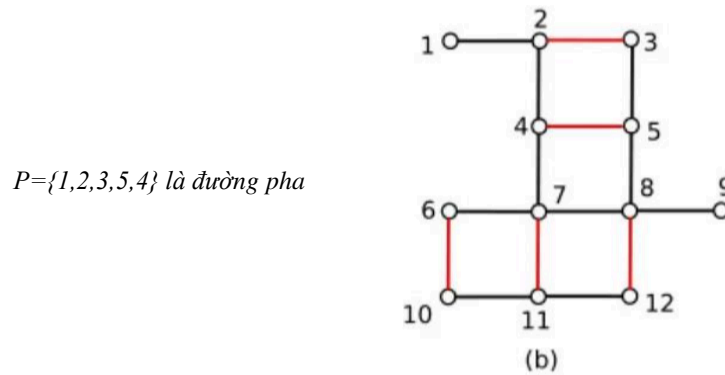
Một đỉnh của đồ thị được gọi là **đã ghép (matched)** nếu nó là đầu mút của một cạnh trong M . Ngược lại, ta gọi đỉnh đó là **đỉnh chưa ghép**.

7.11.2.7. Cặp ghép hoàn hảo (Perfect matching)

Một cặp ghép được gọi là **hoàn hảo** nếu mọi đỉnh của đồ thị đều đã ghép. Một cặp ghép hoàn hảo là một cặp ghép cực đại (nhưng điều ngược lại chưa chắc đã đúng).

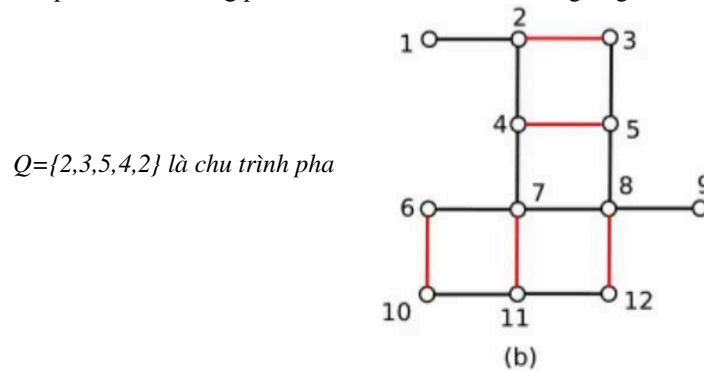
7.11.2.8. Đường pha (alternating path)

Một đường pha của cặp ghép M là một đường đi trên đồ thị sao cho với bất kì hai cạnh liên tục nào trên đường đi, đúng một trong hai cạnh thuộc M .



7.11.2.9. Chu trình pha (alternating cycle)

Một chu trình pha là một đường pha mà điểm đầu và điểm cuối giống nhau.



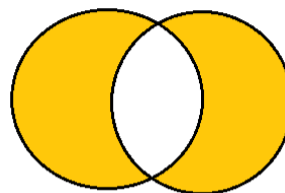
7.11.2.10. Hiệu đối xứng (symmetric difference)

Cho hai tập **A** và **B**, hiệu đối xứng của **A** và **B** kí hiệu là $A \oplus B$ là một tập hợp thỏa mãn:

$$A \oplus B = (A \setminus B) \cup (B \setminus A)$$

Biểu đồ Venn của hiệu đối xứng

Phần bôi màu thể hiện cho hiệu đối xứng của hai tập.



Định lý thể hiện mối liên hệ giữa cặp ghép và đường pha.

Nếu **M** là một cặp ghép của đồ thị $G(V, E)$ và **P** là đường pha, trong đó mỗi đỉnh của **P** hoặc là đỉnh chưa ghép, hoặc là đỉnh đã ghép của **M** thuộc **P**, thì $M \oplus P$ cũng là cặp ghép của $G(V, E)$.

7.11.2.11. Đường tăng (augmenting path)

Một đường pha được gọi là một **đường tăng** nếu nó có điểm đầu và điểm cuối là hai đỉnh chưa ghép.

Ví dụ các cạnh (2,3) và (4,5) thuộc cặp ghép, lúc đó đường pha $P = \{1, 2, 3, 4, 5, 6\}$ là đường tăng.



7.11.3. Thuật toán tìm cặp ghép

7.11.3.1. Thuật toán ghép cặp của Edmonds

7.11.3.1.1. Ý tưởng thuật toán

Tìm cặp ghép cực đại $(G(V, E))$:

$M = \emptyset$

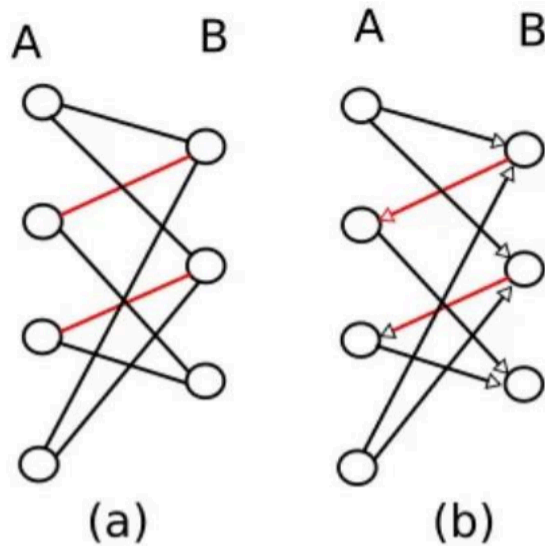
While (còn tồn tại đường tăng **P** trong **G**)

$M = M \oplus P$

Return **M**.

*Vấn đề đặt ra là làm sao tìm đường tăng **P** trong **G** ?*

Để dễ hình dung ta quy bài toán tìm đường tăng về bài toán tìm đường trong đồ thị có hướng; bài toán này về mặt trực quan có vẻ dễ hơn. Một phương pháp đơn giản nhất để tìm đường là lần lượt sử dụng BFS (hoặc DFS), bắt đầu từ một đỉnh chưa ghép của **A**, tìm đường tới một đỉnh chưa ghép của **B**.



Hình (a) mô tả đồ thị ghép cặp của hai tập A, B

Hình (b) chuyển từ đồ thị (a) sang đồ thị có hướng.

7.11.3.1.2. Code tham khảo

```
#include <bits/stdc++.h>
const int N = 1010;
using namespace std;
int matchX[N], matchY[N], Q[N], T[N];
int m, n, e;
vector<int> a[N];

int BFS() {
    int l = 1, r = 0, i, u, v;
    for(i = 1; i <= m; i++)
        if (matchX[i] == 0) Q[++r] = i;
    for(i = 1; i <= n; i++) T[i] = 0;
    while (l <= r) {
        u = Q[l++];
        for(i = 0; i < a[u].size(); i++) {
            v = a[u][i];
            if (T[v] == 0) {
                T[v] = u;
                if (matchY[v] == 0) return v;
                else Q[++r] = matchY[v];
            }
        }
    }
    return 0;
}

void Enlarge(int y) {
    int next, x;
    for(; y; y = next) {
        x = T[y];
        next = matchX[x];
        matchX[x] = y;
    }
}
```

```

        matchY[y] = x;
    }
}

int main() {
    freopen("capghep.inp", "r", stdin);
    freopen("capghep.out", "w", stdout);

    scanf("%d %d %d", &m, &n, &e);
    int i, u, v;
    for(i = 1; i <= e; i++) {
        scanf("%d %d", &u, &v);
        a[u].push_back(v);
    }
    while (i = BFS()) Enlarge(i);
    int res = 0;
    for(i = 1; i <= m; i++) if (matchX[i] != 0) res++;
    printf("%d\n", res);
}

```

7.11.3.1.3. Đánh giá độ phức tạp

Độ phức tạp thuật toán $O(V^3)$

7.11.3.2. Thuật toán Hopcroft–Karp

7.11.3.2.1. Ý tưởng thuật toán

Ý tưởng trung tâm của thuật toán Hopcroft-Karp là mỗi lần tìm đường tăng bằng duyệt đồ thị, thay vì tìm một đường tăng, ta tìm một tập các đường tăng rời nhau (vertex-disjoint), kí hiệu là P . Tập đường tăng rời nhau này có tính chất:

- Tối đại (maximal): mọi đường tăng khác đều có chung ít nhất một đỉnh với P .
- Ngắn nhất: mỗi đường tăng trong P là một đường tăng có chiều dài ngắn nhất so với tất cả các đường tăng của cặp ghép hiện tại M .

7.11.3.2.2. Code tham khảo

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

struct matcher {
    const int oo = 100000000;
    int m, n;
    vector<int> mx, my, dist;
    vector<vector<int>> ke;
    int matched;

    matcher(int m, int n):
        m(m), n(n),
        mx(m+1, 0), my(n+1, 0), dist(m+1),
        ke(m+1),
        matched(0) {}

    void add_edge(int u, int v) {
        ke[u].push_back(v);
    }

    bool bfs() {
        queue<int> Q;
    }
}

```

```

        for (int u=1; u<=m; u++) {
            if (!mx[u]) {
                dist[u] = 0;
                Q.push(u);
            } else {
                dist[u] = oo;
            }
        }

        bool found = false;
        while (!Q.empty()) {
            int u = Q.front(); Q.pop();
            for (int v: ke[u]) {
                if (!my[v]) {
                    found = true;
                } else if (dist[my[v]] == oo) {
                    dist[my[v]] = dist[u] + 1;
                    Q.push(my[v]);
                }
            }
        }

        return found;
    }

    bool dfs(int u) {
        if (dist[u] == oo) return false;
        for (int v: ke[u]) {
            if (!my[v] || (dist[my[v]] == dist[u] + 1 && dfs(my[v]))) {
                mx[u] = v;
                my[v] = u;
                return true;
            }
        }
        return false;
    }

    void match() {
        while(bfs()) {
            for (int u=1; u<=m; u++) if (!mx[u]) matched += dfs(u);
        }
    }
};

int main() {

    freopen("capghep.inp", "r", stdin);
    freopen("capghep.out", "w", stdout);

    ios::sync_with_stdio(false); cin.tie(0);
    int m, n, k; cin >> m >> n >> k;
    matcher z(m, n);
    while (k--) {
        int u, v;
        cin >> u >> v;
        z.add_edge(u, v);
    }
    z.match();
    cout << z.matched;
    return 0;
}

```

7.11.3.2.3. Đánh giá độ phức tạp

Độ phức tạp thuật toán $O(E\sqrt{V})$

7.11.5. Bài toán tìm cặp ghép cực đại có trọng số cực tiểu

7.11.5.1. Phát biểu bài toán

Cho đồ thị hai phía $G = (X \cup Y, E)$; Các đỉnh của X ký hiệu là $x_1, x_2, \dots, x_n$, các đỉnh của Y ký hiệu là $y_1, y_2, \dots, y_n$. Mỗi cạnh của G được gán một trọng số không âm. Một bộ ghép đầy đủ trên G là một tập n cạnh thuộc E đôi một không có đỉnh chung. Trọng số của bộ ghép là tổng trọng số các cạnh nằm trong bộ ghép.

Input: dữ liệu vào từ file **hungarian.inp**

- Dòng 1: Chứa số n ($1 \leq n \leq 200$)
- Các dòng tiếp theo, mỗi dòng chứa 3 số nguyên i, j, c cho biết có một cạnh (x_i, y_j) và trọng số cạnh đó là c ($0 \leq c \leq 200$).

Output: Ghi ra file **hungarian.out**

- Dòng 1: Ghi trọng số bộ ghép tìm được
- n dòng tiếp, mỗi dòng ghi hai số (u, v) tương trưng cho một cạnh (x_u, y_v) được chọn vào bộ ghép.

Ví dụ

hungarian.inp	hungarian.out
4	3
1 1 0	1 1
1 2 0	2 4
2 1 0	3 2
2 4 2	4 3
3 2 1	
3 3 0	
4 3 0	
4 4 9	

7.11.5.2. Thuật toán Hungari

```
//code by ladpro98
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

struct BipartiteGraph {
    vector<vector<int>> > c; // cost matrix
    vector<int> fx, fy; // potentials
    vector<int> matchX, matchY; // corresponding vertex
    vector<int> trace; // last vertex from the left side
    vector<int> d, arg; // distance from the tree && the corresponding node
    queue<int> Q; // queue used for BFS

    int n; // assume that |L| = |R| = n
    int start; // current root of the tree
    int finish; // leaf node of the augmenting path

    BipartiteGraph(int n) {
```

```

    this->n = n;
    c = vector<vector<int>> >(n + 1, vector<int>(n + 1, INF));
    fx = fy = matchX = matchY = trace = d = arg = vector<int>(n + 1);
}

void addEdge(int u, int v, int cost) { c[u][v] = min(c[u][v], cost); }
int cost(int u, int v) { return c[u][v] - fx[u] - fy[v]; }

void initBFS(int root) {
    start = root;
    Q = queue<int>(); Q.push(start);
    for (int i = 1; i <= n; ++i) {
        trace[i] = 0;
        d[i] = cost(start, i);
        arg[i] = start;
    }
}

int findPath() {
    while (!Q.empty()) {
        int u = Q.front(); Q.pop();
        for (int v = 1; v <= n; ++v) if (trace[v] == 0) {
            int w = cost(u, v);
            if (w == 0) {
                trace[v] = u;
                if (matchY[v] == 0) return v;
                Q.push(matchY[v]);
            }
            if (d[v] > w) d[v] = w, arg[v] = u;
        }
    }
    return 0;
}

void enlarge() {
    for (int y = finish, next; y; y = next) {
        int x = trace[y];
        next = matchX[x];
        matchX[x] = y;
        matchY[y] = x;
    }
}

void update() {
    int delta = INF;
    for (int i = 1; i <= n; ++i)
        if (trace[i] == 0) delta = min(delta, d[i]);
    fx[start] += delta;
    for (int i = 1; i <= n; ++i) {
        if (trace[i] != 0) {
            fx[matchY[i]] += delta;
            fy[i] -= delta;
        } else {
            d[i] -= delta;
            if (d[i] == 0) {
                trace[i] = arg[i];
                if (matchY[i] == 0)
                    finish = i;
                else
                    Q.push(matchY[i]);
            }
        }
    }
}

```

```

    }
}

void hungarian() {
    for (int i = 1; i <= n; ++i) {
        initBFS(i);
        do {
            finish = findPath();
            if (finish == 0) update();
        } while (finish == 0);
        enlarge();
    }
}

void show() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) if (matchX[i]) ans += c[i][matchX[i]];
    cout << ans << endl;
    for (int i = 1; i <= n; ++i) cout << i << ' ' << matchX[i] << endl;
}

};

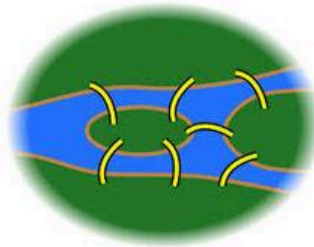
int main() {
    ios::sync_with_stdio(false);
    int n; cin >> n;
    BipartiteGraph G (n);
    int u, v, c;
    while (cin >> u >> v >> c) G.addEdge(u, v, c);
    G.hungarian();
    G.show();
    return 0;
}

```

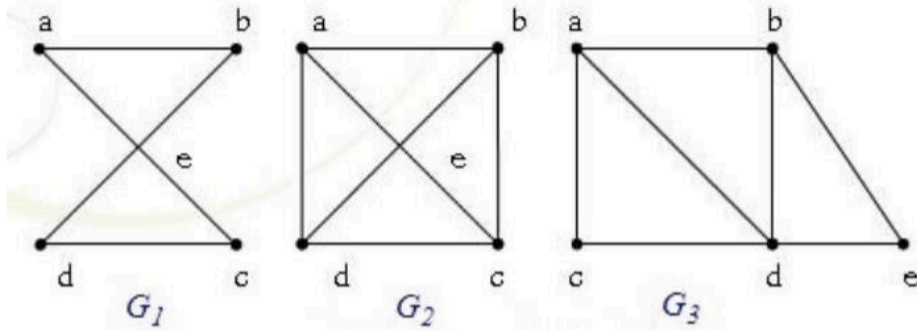
7.12. Đường đi Euler, chu trình Euler

7.12.1. Khái niệm

- Trong lý thuyết đồ thị, một đường đi trong đồ thị $G = (X, E)$ được gọi là **đường đi Euler** nếu nó đi qua tất cả các cạnh của đồ thị, mỗi cạnh đúng một lần.
- Đường đi Euler có đỉnh cuối cùng trùng với đỉnh xuất phát gọi là **chu trình Euler**.
- Đồ thị Euler là đồ thị chứa chu trình Euler
- Đồ thị nửa Euler là đồ thị chứa đường đi Euler
- Khái niệm chu trình Euler xuất phát từ bài toán bảy cây cầu do Euler giải quyết vào khoảng năm 1737. Đường đi Euler có thể tìm thấy trong các bài toán vui vẽ một nét (vẽ một hình nào đó mà không nhấc bút khỏi mặt giấy, không tô lại cạnh nào hai lần).



• Ví dụ



Đồ thị G_1 là đồ thị Euler vì nó có chu trình Euler $a - e - c - d - e - b - a$.

Đồ thị G_2 không có chu trình Euler cũng như đây chuyển Euler.

Đồ thị G_3 không có chu trình Euler nhưng nó có đường đi Euler $a - c - d - e - b - a - d - b$, vì thế G_3 là đồ thị nửa Euler.

7.12.2. Phát biểu bài toán

Cho đơn đồ thị vô hướng liên thông $G = (V, E)$ gồm n đỉnh và m cạnh, các đỉnh được đánh số từ 1 tới n và các cạnh được đánh số từ 1 tới m . Hãy tìm 1 đường đi Euler trên G .

Input: Dữ liệu vào từ file **eulerpath.inp**

- Dòng 1: Chứa hai số n, m .
- M dòng tiếp theo: Dòng thứ i có dạng hai số nguyên u, v . Trong đó u, v là chỉ số hai đỉnh đầu cuối của cạnh thứ i .

Output: Ghi ra file **eulerpath.out**

Gồm 1 dòng duy nhất là 1 dãy các số mô tả các đỉnh trên đường đi Euler

Ví dụ

eulerpath.inp	eulerpath.out	Hình minh họa
8 9 1 2 1 3 4 2 4 3 4 5 4 6 5 7 6 8 7 8	1 3 4 6 8 7 5 4 2 1	

Giới hạn:

- $1 \leq n \leq 100$
- $1 \leq m \leq n(n-1)/2$

7.12.3. Ý tưởng thuật toán

7.12.3.1. Thuật toán Fleury

Xuất phát từ 1 đỉnh nào đó của đồ thị G (G đã loại các đỉnh cô lập) ta đi theo các cạnh của nó một cách tùy ý chỉ cần tuân thủ 2 quy tắc sau:

- Cạnh này không phải là “cầu” (việc bỏ cạnh này không làm cho đồ thị mất liên thông).
- Xóa bỏ cạnh đã đi qua và đồng thời xóa cả những đỉnh cô lập tạo thành.

Nếu không tìm thấy cạnh nào thỏa, thuật toán dừng. Ngược lại, từ đỉnh được nối đến, lặp lại quá trình trên. Một cách tự nhiên, chu trình Euler sẽ quay lại điểm bắt đầu, đường đi Euler sẽ đến điểm kết thúc.

Lưu ý: Khi xóa bỏ cạnh đi qua và đỉnh cô lập, sẽ hình thành một đồ thị mới. Việc xét “cầu” tiếp theo sẽ trên đồ thị mới này.

Nhận xét: Thuật toán này tuy đơn giản nhưng thường không được sử dụng bởi việc xác định “cầu” trong đồ thị không phải đơn giản.

7.12.3.2. Thuật toán tìm chu trình Euler dựa trên Stack

Định lý:

Một đồ thị vô hướng liên thông $G=(V,E)$ khi và chỉ khi mọi đỉnh của nó đều có bậc chẵn.

Chứng minh: tham khảo tài liệu giáo khoa chuyên tin quyển 1/207

Việc chứng minh định lý trên cho ta một thuật toán hữu hiệu để chỉ ra chu trình Euler trên đồ thị Euler. Thuật toán này hoặc động dựa trên một ngăn xếp Stack và được mô tả cụ thể như sau:

- Bắt đầu từ đỉnh 1 ta đi thoải mái theo các cạnh của đồ thị cho đến khi không đi được nữa, đi tới đỉnh nào ta đẩy đỉnh đó vào ngăn xếp và đi qua cạnh nào ta xóa cạnh đó khỏi đồ thị.
- Khi không đi được nữa thì ngăn xếp sẽ chứa các đỉnh trên một chu trình C bắt đầu và kết thúc ở đỉnh 1. Sau đó chúng ta lấy lần lượt các đỉnh ra khỏi ngăn xếp tương đương với việc đi ngược chu trình C .
- Nếu đỉnh được lấy ra (u) không có cạnh nào còn lại liên thuộc với nó thì u sẽ được ghi ra chu trình Euler, ngược lại, nếu u vẫn còn có cạnh liên thuộc thì ta lại đi tiếp từ u theo cách trên và đẩy thêm vào ngăn xếp một chu trình C' bắt đầu và kết thúc tại u .
- Khi lấy các đỉnh ra khỏi ngăn xếp sẽ tương đương với việc đi ngược lại chu trình C' rồi tiếp tục đi ngược phần còn lại của chu trình C trong ngăn xếp... Có thể hình dung là thuật toán lần ngược chu trình C khi đến đỉnh u thì thay u bằng cả một chu trình C' ...

7.12.4. Code tham khảo

```
#include <bits/stdc++.h>

using namespace std;

const int nm=102;

int n,m,a[nm][nm];
int ns,st[nm*nm];
```

```

int bac[nm];

void nhap()
{
    scanf("%d%d", &n, &m);
    int i, u, v;
    for(i=1; i<=m; ++i)
    {
        scanf("%d%d", &u, &v);
        a[u][v]=a[v][u]=1;
        bac[u]++; bac[v]++;
    }
}

void xuli()
{
    int i, j;
    for(i=1; i<=n; ++i) if (bac[i]%2) break;
    ns=1;
    if (i<=n) st[1]=i; else st[1]=1;
    while (ns)
    {
        i=st[ns];
        for(j=1; j<=n; ++j)
        {
            if (a[i][j])
            {
                st[++ns]=j; a[i][j]--; a[j][i]--;
                break;
            }
        }
        if (i==st[ns])
        {
            printf("%d ", i); ns--;
        }
    }
}

int main()
{
    freopen("eulerpath.inp", "r", stdin);
    freopen("eulerpath.out", "w", stdout);

    nhap(); xuli();
}

```

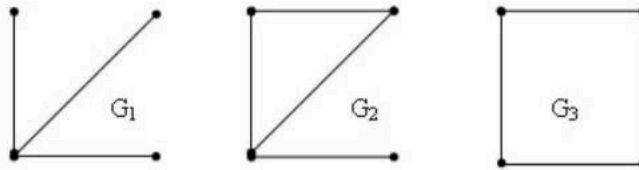
7.12.5. Đánh giá độ phức tạp

Độ phức tạp thuật toán $O(V \cdot E)$

7.13. Đường đi Halminton, chu trình Halminton

7.13.1. Khái niệm

- **Đường đi Hamilton** là đường đi qua các đỉnh của đồ thị và đi qua mỗi đỉnh đúng 1 lần.
- **Chu trình Hamilton** là đường đi Hamilton xuất phát từ một đỉnh, đi qua tất cả các đỉnh khác của đồ thị, mỗi đỉnh đúng một lần và quay trở về nơi xuất phát.
- **Đồ thị Hamilton** là đồ thị có chứa ít nhất một chu trình Hamilton.
- **Đồ thị nửa Hamilton** là đồ thị có chứa đường đi Hamilton



G1: không phải là nửa Hamilton

G2: là đồ thị nửa Hamilton

G3: là đồ thị Hamilton

7.13.2. Phát biểu bài toán

Cho đồ thị vô hướng có N đỉnh và M cạnh, bạn hãy kiểm tra xem đồ thị đã cho có chứa đường đi Hamilton hay không?

INPUT: Dữ liệu vào từ file **hamiltonpath.inp** gồm

- Dòng đầu tiên gồm hai số nguyên n, m .
- M dòng tiếp theo, mỗi dòng chứa hai số nguyên x, y tương ứng là có cạnh nối giữa hai đỉnh x, y .

OUTPUT: Ghi ra file **hamiltonpath.out** “YES” nếu đồ thị có chứa đường đi Hamilton, ngược lại, in ra “NO”.

RÀNG BUỘC

- $1 \leq N \leq 10$
- $1 \leq M \leq 10$
- $1 \leq X, Y \leq N$

VÍ DỤ

Hamiltonpath.inp	Hamiltonpath.out
4 3 0 1 1 2 1 3	NO

7.13.3. Ý tưởng thuật toán

Hiện nay người ta vẫn chưa tìm ra thuật toán với độ phức tạp đa thức để tìm chu trình Hamilton cũng như đường đi Hamilton trong trường hợp đồ thị tổng quát. Tất cả các thuật toán tìm chu trình Hamilton hiện nay đều dựa trên mô hình duyệt, có thể kết hợp với một số mẹo cài đặt. Code tham khảo bên dưới là một trong những cách cài đặt duyệt hết các trường hợp.

7.13.4. Code tham khảo

```
//Code by Nguyen Quang Thuan
#include <bits/stdc++.h>
#define NMAX 10
#define FOR(i,aa,bb) for(int i=(aa);i<=(bb);i++)
using namespace std;

//ma tran ke luu do thi
bool a[NMAX][NMAX];
```

```

//so dinh va so canh cua do thi
int n,m;
//luu ket qua chu trinh
int path[NMAX];
//mang danh dau dinh da duoc tham chua
bool freenode[NMAX];
//Cờ đánh dấu tồn tại chu trình Hamilton
bool flag;

//Thu tuc thu cho nut thu i
int trynode(int i){
    FOR(k,1,n){
        if (freenode[k] && (a[path[i-1]][k])){
            //chọn đỉnh k
            path[i]=k;
            //cout << k << "--";
            //Nếu đã tìm được chu trình thì kết thúc
            if (i==n)
            {
                if (a[k][path[1]])
                {
                    flag=true;
                    return 0;
                }
            }
            else
            {
                freenode[k] = false;
                trynode(i+1);
                freenode[k] = true;
            }
        }
    }
    return 0;
}
int main()
{
    freopen("hamiltonpath.inp","r",stdin);
    freopen("hamiltonpath.out","w",stdout);
    int x,y;
    //doc du lieu
    cin >> n >> m;

    //khởi tạo
    memset(a,sizeof(a),0);
    FOR(i,1,n) freenode[i]=true;
    FOR(i,1,m){
        cin >> x >> y;
        a[x+1][y+1] = true;
        a[y+1][x+1] = true;
    }

    //Thử từng nút
    path[1]=1;freenode[1]=false;
    trynode(2);

    //Hiển thị kết quả
    //cout << flag << endl;
    if (flag) cout << "YES";
    else cout << "NO";
    return 0;
}

```

7.13.5. Đánh giá độ phức tạp:

Độ phức tạp của thuật toán duyệt vết trên xấp xỉ $O(n!)$

7.13.6. Bài toán minh họa

Cho đơn đồ thị vô hướng liên thông $G = (V, E)$ gồm n đỉnh và m cạnh, các đỉnh được đánh số từ 1 tới n và các cạnh được đánh số từ 1 tới m . Hãy tìm 1 chu trình Hamilton trên G

Input: Dữ liệu vào từ file **hamiltoncycle.inp**

- Dòng 1: Chứa hai số n, m .
- M dòng tiếp theo: Dòng thứ i có dạng hai số nguyên u, v . Trong đó u, v là chỉ số hai đỉnh đầu mút của cạnh thứ i .

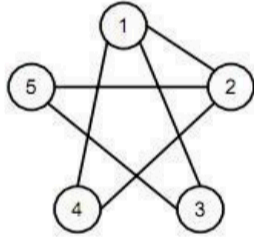
Output: Ghi ra file **hamiltoncycle.out**

Gồm 1 dòng duy nhất là 1 dãy các số mô tả các đỉnh trên chu trình Hamilton

Ràng buộc

- $1 \leq n \leq 10$
- $1 \leq m \leq n(n-1)/2$

Ví dụ

hamiltoncycle.inp	hamiltoncycle.out	Hình minh họa
5 6 1 2 1 3 2 4 3 5 4 1 5 2	1 4 2 5 3 1	

Code tham khảo

```
//Code by Nguyen Quang Thuan
#include <bits/stdc++.h>
#define NMAX 10
#define FOR(i,aa,bb) for(int i=(aa);i<=(bb);i++)
using namespace std;

//ma tran ke luu do thi
bool a[NMAX][NMAX];
//so dinh va so canh cua do thi
int n,m;
//luu ket qua chu trinh
int path[NMAX];
//mang danh dau dinh da duoc tham chua
bool freenode[NMAX];
//Cờ đánh dấu tồn tại chu trình Hamilton
bool flag;
```

```

//Thu tục thu cho nút thu i
int trynode(int i){
    FOR(k,1,n){
        if (freenode[k] && (a[path[i-1]][k])){
            //chọn đỉnh k
            path[i]=k;
            //cout << k << "--";
            //Nếu đã tìm được chu trình thì kết thúc
            if (i==n)
            {
                if (a[k][path[1]])
                {
                    flag=true;
                    return 0;
                }
            }
        }
        else
        {
            freenode[k] = false;
            trynode(i+1);
            freenode[k] = true;
        }
    }
}
return 0;
}
int main()
{
    freopen("hamiltoncycle.inp","r",stdin);
    freopen("hamiltoncycle.out","w",stdout);
    int x,y;
    //doc du lieu
    cin >> n >> m;

    //khởi tạo
    memset(a,sizeof(a),0);
    FOR(i,1,n) freenode[i]=true;
    FOR(i,1,m){
        cin >> x >> y;
        a[x][y] = true;
        a[y][x] = true;
    }

    //Thử từng nút
    path[1]=1;freenode[1]=false;
    trynode(2);

    //Hiển thị kết quả
    if (flag){
        FOR(i,1,n) cout << path[i] << " ";
        cout << path[1];
    }
    return 0;
}

```

Mục lục

PHẦN 7: ĐỒ THỊ.....	1
7.1. Khái niệm cơ bản.....	1
7.2. Phân loại đồ thị.....	1
7.3. Biểu diễn đồ thị.....	2
7.3.1. Ma trận kề.....	2
7.3.1.1 Biểu diễn đồ thị vô hướng.....	2
7.3.1.1.1 Hình minh họa.....	2
7.3.1.1.2 mtkvohuong – Ma trận kề vô hướng.....	3
7.3.1.2 Biểu diễn đồ thị có hướng.....	4
7.3.1.2.1. Hình minh họa.....	4
7.3.1.2.2. mtkcohuong – Ma trận kề có hướng.....	4
7.3.2. Danh sách kề.....	5
7.3.2.1. Biểu diễn đồ thị vô hướng.....	5
7.3.2.1.1 Hình minh họa.....	5
7.3.2.1.2 dskvohuong – Danh sách kề vô hướng.....	5
7.3.2.2. Biểu diễn đồ thị có hướng.....	6
7.3.2.2.1 Hình minh họa.....	6
7.3.2.2.1 dskcohuong – Danh sách kề có hướng.....	7
7.4. Phương pháp duyệt đồ thị.....	8
7.4.1. Duyệt rộng.....	8
7.4.1.1. Khái niệm.....	8
7.4.1.2. Bài tập vận dụng.....	8
7.4.1.2.1. bfs – Duyệt rộng.....	8
7.4.1.2.2. levelnode.....	10
7.4.1.2.3. Bisland.....	12
7.4.1.2.4. obattle.....	13
7.4.1.2.5. HPARTY.....	15
7.4.2. Duyệt sâu.....	17
7.4.2.1. Khái niệm.....	17
7.4.2.2. Bài tập vận dụng.....	17
7.4.2.2.1.dfs – duyệt sâu.....	17
7.4.2.2.2. unreachable – Nút không thể đến được.....	18

7.4.2.2.3. TFRIENDS - THÀNH VÀ BẠN GÁI	21
7.4.2.2.4. HVERTEX – ĐỈNH HẠNH PHÚC.....	23
7.4.2.2.5. MNODES - Minimum Nodes	24
7.4.2.2.6. SSEE – SIGHTSEEING.....	26
7.4.2.2.7. MEDGES – SỐ CẠNH LỚN NHẤT.....	28
7.5 Đường đi ngắn nhất trên đồ thị.....	30
7.5.1. Bài tập cơ bản.....	30
7.5.1.1. Phát biểu bài toán	30
7.5.1.2. Ý tưởng	31
7.5.1.3. Thuật toán Bellman -Ford.....	31
7.5.1.3.1. ý tưởng	31
7.5.1.3.2. Code tham khảo.....	31
7.5.1.3.3. Một số lưu ý	32
7.5.1.4. Thuật toán Dijkstra.....	33
7.5.1.4.1. Ý tưởng	33
7.5.1.4.2. Code tham khảo.....	33
7.5.1.4.3. Một số lưu ý	34
7.5.2. Bài tập vận dụng.....	34
7.5.2.1. MCITY – TÌM ĐƯỜNG ĐI NGẮN NHẤT.....	34
7.5.2.2. MHIGH – CHI PHÍ TỐI THIỂU ĐỂ LÀM BẰNG ĐƯỜNG ĐI	36
7.5.2.3. MALPHA - Ma trận chữ cái	38
7.6. Cây khung và cây khung nhỏ nhất.....	40
7.6.1. Một số khái niệm cơ bản.....	40
7.6.2. Bài tập cơ bản.....	40
7.6.2.1. Phát biểu bài toán - MST – MINIMUM SPANNING TREE.....	40
7.6.2.2. Thuật toán Kruskal	40
7.6.2.2.1. Giới thiệu	40
7.6.2.2.2. Tư tưởng thuật toán.....	40
7.6.2.2.3. Mô phỏng thuật toán.....	41
7.6.2.2.4. Code tham khảo.....	44
7.6.2.2.5. Đánh giá độ phức tạp.....	45
7.6.2.3. Thuật toán Prim.....	45
7.6.2.3.1. Giới thiệu	45

7.6.2.3.2. Tư tưởng thuật toán.....	45
7.6.2.3.3. Mô phỏng thuật toán.....	45
7.6.2.3.4. Code tham khảo.....	48
7.6.3. Bài tập vận dụng.....	49
7.6.3.1. MAXST – CÂY KHUNG CÓ TRỌNG SỐ LỚN NHẤT	49
7.6.3.2. SNETWORK – SIMULATE NETWORK.....	51
7.6.3.3. TUỔI NƯỚC	53
7.7. Thành phần liên thông	55
7.7.1. Khái niệm.....	55
7.7.2. Phát biểu bài toán – tarjan - Tìm TPLT mạnh	56
7.7.3. Thuật toán tarjan.....	56
7.7.4 Code tham khảo thuật toán tarjan.....	57
7.7.5. Đánh giá độ phức tạp.....	58
7.8. Khớp và cầu.....	58
7.8.1. Khái niệm.....	58
7.8.2. Bài toán định chiều đồ thị	58
7.8.3. Bài toán đếm số lượng khớp, cầu	59
7.8.4. Ý tưởng	59
7.8.5. Code tham khảo bài toán đếm số lượng khớp cầu	60
7.8.6. Đánh giá độ phức tạp về thời gian:.....	61
7.9. Sắp xếp tô pô	61
7.9.1. Khái niệm.....	61
7.9.2. Phát biểu bài toán	61
7.9.3. Ý tưởng thuật toán.....	62
7.9.4. Code tham khảo.....	62
7.9.4.1. Vận dụng giải thuật duyệt sâu.....	62
7.9.4.2. Vận dụng giải thuật duyệt rộng.....	63
7.9.5. Đánh giá độ phức tạp thuật toán.....	64
7.10. Luồng cực đại	64
7.10.1. Khái niệm.....	64
7.10.1.1. Mạng (flow network).....	64
7.10.1.2. Luồng (flow)	64
7.10.1.3. Một số khái niệm khác.....	64

7.10.1.4. Mạng thặng dư (Residual network).....	65
7.10.1.5. Đường tăng luồng (augmenting path)	65
7.10.2. Phát biểu bài toán	66
7.10.3. Ý tưởng thuật toán Ford-Fulkerson	66
7.10.4. Code tham khảo.....	67
7.10.4.1. Thuật toán Edmond-Karp	67
7.10.4.1. Thuật toán Dinic:.....	68
7.11. Cặp ghép.....	70
7.11.1. Phát biểu bài toán	70
7.11.2. Một số khái niệm	71
7.11.2.1. Đồ thị hai phía.....	71
7.11.2.2. Kiểm tra một đồ thị có phải là đồ thị hai phía hay không	71
7.11.2.2.1. Phát biểu bài toán – cbipartite.....	71
7.11.2.2.2. Code tham khảo.....	72
7.11.2.2.2. Đánh giá độ phức tạp.....	73
7.11.2.3. Cặp ghép	73
7.11.2.4. Cặp ghép tối đại (Maximal matching).....	73
7.11.2.5. Cặp ghép cực đại (Maximum matching)	74
7.11.2.6. Đỉnh đã ghép, đỉnh chưa ghép.....	74
7.11.2.7. Cặp ghép hoàn hảo (Perfect matching)	74
7.11.2.8. Đường pha (alternating path)	74
7.11.2.9. Chu trình pha (alternating cycle).....	74
7.11.2.10. Hiệu đối xứng (symmetric difference)	75
7.11.2.11. Đường tăng (augmenting path)	75
7.11.3. Thuật toán tìm cặp ghép.....	75
7.11.3.1. Thuật toán ghép cặp của Edmonds.....	75
7.11.3.1.1. Ý tưởng thuật toán.....	75
7.11.3.1.2. Code tham khảo.....	76
7.11.3.1.3. Đánh giá độ phức tạp.....	77
7.11.3.2. Thuật toán Hopcroft–Karp.....	77
7.11.3.2.1. Ý tưởng thuật toán.....	77
7.11.3.2.2. Code tham khảo.....	77
7.11.3.2.3. Đánh giá độ phức tạp.....	79

7.11.5. Bài toán tìm cặp ghép cực đại có trọng số cực tiểu.....	79
7.11.5.1. Phát biểu bài toán	79
7.11.5.2. Thuật toán Hungari.....	79
7.12. Đường đi Euler, chu trình Euler	81
7.12.1. Khái niệm	81
7.12.2. Phát biểu bài toán	82
7.12.3. Ý tưởng thuật toán	83
7.12.3.1. Thuật toán Fleury	83
7.12.3.2. Thuật toán tìm chu trình Euler dựa trên Stack	83
7.12.4. Code tham khảo	83
7.12.5. Đánh giá độ phức tạp	84
7.13. Đường đi Halminton, chu trình Halminton.....	84
7.13.1. Khái niệm	84
7.13.2. Phát biểu bài toán	85
7.13.3. Ý tưởng thuật toán	85
7.13.4. Code tham khảo	85
7.13.5. Đánh giá độ phức tạp:.....	87
7.13.6. Bài toán minh họa.....	87